

The Nanz Language Book — v7.0

Table of Contents

Chapter 1: What is Nanz?

- 1.1 The Compilation Pipeline
- 1.2 Nanz vs. MinZ
- 1.3 Nanz vs. PL/M-80
- 1.4 Design Philosophy

Chapter 2: Syntax Reference

- 2.1 Module Structure
- 2.2 Functions
- 2.3 Variables
- 2.4 Global Variables
- 2.5 Control Flow
- 2.6 Pointers and Arrays
- 2.7 Structs
- 2.8 Lambdas
- 2.9 Casts
- 2.10 Extern Functions
- 2.11 Register Annotations on Parameters
- 2.12 Ranged Types
- 2.13 Interfaces and UFCS
- 2.14 Inline Assembly
- 2.15 ptr() — Direct Memory Access
- 2.16 |> Value Pipe Operator
- 2.17 sizeof(Type)

Chapter 3: Type System

- 3.1 Numeric Types
- 3.2 Signed Arithmetic
- 3.3 Pointer Types
- 3.4 The u32 “ClassDWord” via EXX Shadow Pair
- 3.5 Calling Convention

Chapter 4: Structs, Methods, and Interfaces

- 4.1 Struct Declaration and Layout
- 4.2 Methods and UFCS
- 4.3 Interfaces and Static Dispatch
- 4.4 impl Blocks (v7.0)
- 4.5 Operator Overloading

Chapter 5: Iterator Chains

- 5.1 Combinators
- 5.2 Fusion
- 5.3 Closure Capture

5.4 E2E Verification	
Chapter 6: <code>range(lo..hi)</code> — Counter-Based Iteration	
6.1 Overview	
6.2 Counting Semantics	
6.3 The Parallel Copy Bug: Caught by Dual-VM	
Chapter 7: Compile-Time Assertions and Sandbox Blocks	
7.1 The Idea	
7.2 Dual-VM Execution	
7.3 What Each VM Catches	
7.4 Syntax Variants	
7.5 Multi-Value Assertions	
7.6 How Z80 Asserts Work	
7.7 Sandbox Blocks: Shared VM State	
Chapter 8: The Optimization Pipeline	
8.1 Overview of Passes	
8.2 LUT Generation	
8.3 BranchEquiv	
8.4 CondRetSink and Flag Fusion: <code>abs_diff</code> in 4 Instructions	
8.5 PreallocCoalesce: <code>DJNZ</code> from 5 Instructions to 1	
8.6 Trivial Inliner	
8.7 Interprocedural Contract Optimization (PFCCO)	
8.8 Multiply Strength Reduction	
Chapter 9: Multiple Compilation Targets	
9.1 The Backend Spectrum	
9.2 MOS 6502 Backend	
9.3 <code>MIR2→C</code> : Verification Backend	
9.4 <code>MIR2→QBE</code> : Native Backend	
9.5 The <code>mzn</code> Native Compiler	
9.6 The Big Picture: Four Verification Targets	
9.7 Z80 Platform Targets	
Chapter 10: Z80 Extern and Register Contracts	
10.1 Basic <code>@extern</code>	
10.2 RST Optimization	
10.3 Annotated <code>@extern</code> : Precise Register Contracts	
10.4 The ABI Is Not Fixed	
Chapter 11: Verified Codegen — Showcase	
11.1 <code>abs_diff</code> : Five Passes to Four Instructions	
11.2 <code>sum_range</code> : One Instruction per Iteration	
11.3 <code>mapInPlace</code> : <code>DJNZ</code> Direct (PreallocCoalesce)	
11.4 <code>swap</code> : Zero Instructions	
11.5 Iterator Chain: Zero <code>CALL</code> Overhead	
11.6 <code>add32</code> : 32-bit on an 8-bit CPU	
11.7 <code>GCD</code> : While Loop with Modulo	
11.8 Full Showcase: 24/24 PASS	

Chapter 12: Self-Modifying Code: @smc

12.1 The Concept

12.2 @smc Parameters

12.3 Compiled Sprites

Chapter 13: Native Compilation: mzn

13.1 Overview

13.2 Usage

13.3 VSCode Integration

13.4 Why Both Backends?

Chapter 14: Roadmap — What's Coming

Planned Features

Architecture Aspirations

Chapter 15: Memory Management: Arena Allocators

15.1 The Arena Pattern

15.2 Arena API

15.3 arena_split: Chaining Arenas

15.4 Lifetime Tiers

15.5 Typed Allocation with sizeof

15.6 Generated Z80 Code

15.7 Testing with Sandbox

Chapter 16: Enums, ADTs, Match, and Type Aliases

16.1 Simple Enums

16.2 ADT Enums (Algebraic Data Types)

16.3 Match Expressions

16.4 Match with Payload Binding

16.5 Type Aliases

16.6 Design: Why u16 for ADTs?

Chapter 17: Module System

17.1 Import Styles

17.2 Module Resolution

17.3 Implementation

Chapter 18: Strings and Text Output

18.1 Three String Types

18.2 Z80 String Encoding

18.3 @print and Console Output

18.4 String Interpolation

Chapter 19: Pipe/Trans — Named Iterator Pipelines

19.1 The Problem

19.2 Pipe Declarations

19.3 Applying Pipes

19.4 Zero-Cost Fusion

19.5 |> Prefix Syntax

19.6 Available Stages

19.7 Design Notes

Chapter 20: Metaprogramming — @derive and Introspection

20.1 The Problem

20.2 Native @derive Metafunctions

20.3 How It Works: The MetaRuntime

20.4 VM-Hosted Metafunctions

20.5 Testing @derive

20.6 Design Philosophy

Chapter 21: Cross-Language Imports

21.1 Five Languages, One Pipeline

21.2 Why Five Frontends?

21.3 Importing Lanz Modules

21.4 Importing Lizp Modules

21.5 Importing PL/M-80 Modules

21.6 Importing Pascal Modules

21.7 Module Resolution by Extension

21.8 Safety: Circular Import Detection

21.9 Universal Compile-Time Assertions

21.10 Transpilation Between Frontends

21.11 Why Cross-Language Matters

21.12 Full Test Coverage

Appendix A: Grammar

Appendix B: Register Classes

MIR2 Register Classes

Physical Register Availability

Memory-Backed Registers (\$F0xx)

Appendix C: CLI and Tools

Compiling Nanz

The Toolchain

Running Tests

Appendix D: What's New in v4

Assembly improvements (v3 → v4)

Appendix E: What's New in v4.1

New syntax summary (v4 → v4.1)

Appendix F: What's New in v5

New syntax summary (v4.1 → v5)

Feature gap closed

Appendix G: What's New in v5.2

New syntax summary (v5 → v5.2)

Tetris: Complete Game in Nanz

Appendix H: What's New in v5.3

Five Frontends, One Pipeline

Pascal Hello World on CP/M

Appendix I: What's New in v5.4

FAT12/16 Filesystem Library — `stdlib/fs/fat12.minz`

Chapter 22: Self-Hosting — Can Nanz Compile Itself?

22.1 The Staged Pipeline Vision

22.2 What's Feasible Today

22.3 What's Hard

22.4 The MZV Path

22.5 TinyNanz: A Bootstrapping Subset

22.6 The Multi-Tool Architecture

22.7 Error Handling: @error vs ADT Result

22.8 Roadmap to Self-Hosting

Appendix J: What's New in v7.0

VIR: Z3 SMT Backend (Default)

impl Blocks

Wider Integer Types

Local Array Literals

else-if Chains

Stream Abstraction (`stdlib/core/stream.nanz`)

S-Expression Frontends: Lambda, Match, Let-in

MinZ Corpus Convergence

9 Frontends, One Pipeline

Appendix K: What's New in v8.0 (Birthday Marathon)

@error — Z80-Native Error Propagation

VIR as Default Backend

Five Backends — Z80 to GPU

BCD Packed Decimal Types

RLCA Sled — Barrel Shifter (9 bytes)

MZA INCBIN — Binary Data Embedding

Codegen Quality: Z3 vs Hand-Written

GPU-Precomputed Arithmetic (`z80-optimizer v1.0.0`)

1046 Total Assertions

The Nanz Language Book — v7.0

Modern language. Vintage iron. Zero overhead.

Nanz is a statically typed systems language that compiles to Z80 assembly with no runtime, no garbage collector, and no performance tax. Every abstraction — iterators, lambdas, interfaces, ADTs, pattern matching, impl blocks — disappears at compile time and leaves only tight machine code.

Also targets native AMD64 via C99 and QBE, MOS 6502, and eZ80 (Agon Light 2).

Table of Contents

1. [What is Nanz?](#)
 2. [Syntax Reference](#)
 3. [Type System](#)
 4. [Structs, Methods, and Interfaces](#)
 5. [Iterator Chains](#)
 6. [range\(lo..hi\) — Counter-Based Iteration](#)
 7. [Compile-Time Assertions and Sandbox Blocks](#)
 8. [The Optimization Pipeline](#)
 9. [Multiple Compilation Targets](#)
 10. [Z80 Extern and Register Contracts](#)
 11. [Verified Codegen: Showcase](#)
 12. [Self-Modifying Code: @smc](#)
 13. [Native Compilation: mzn](#)
 14. [Roadmap: What's Coming](#)
 15. [Memory Management: Arena Allocators](#)
 16. [Enums, ADTs, Match, and Type Aliases](#)
 17. [Module System](#)
 18. [Strings and Text Output](#)
 19. [Pipe/Trans: Named Iterator Pipelines](#)
 20. [Metaprogramming: @derive and Introspection](#)
 21. [Cross-Language Imports](#)
 22. [Self-Hosting: Can Nanz Compile Itself?](#)
-
- [Appendix A: Grammar](#)
 - [Appendix B: Register Classes](#)
 - [Appendix C: CLI and Tools](#)
 - [Appendix D: What's New in v4](#)
 - [Appendix E: What's New in v4.1](#)
 - [Appendix F: What's New in v5](#)
 - [Appendix G: What's New in v5.2](#)
 - [Appendix H: What's New in v5.3](#)
 - [Appendix J: What's New in v7.0](#)
-

Chapter 1: What is Nanz?

Nanz (.nanz) is the active frontend language of the **MinZ compiler system**. It targets the MIR2 backend — a modern, SSA-like intermediate representation with:

- **VIR backend (default):** Z3 SMT solver for joint instruction selection + register allocation — provably optimal code
- **PBQP fallback:** cost-weighted register allocation for asm-heavy functions
- Interprocedural calling convention optimization (PFCCO / Z3-PFCCO)
- Pre-allocation coalescing (block-parameter register unification)
- LUT synthesis (pure functions with bounded inputs → lookup tables)
- Compile-time assertion evaluation on both the MIR2 VM *and* the Z80 binary
- A Z80 emulator used as a constant evaluator *inside* the compiler
- Multiple backends: Z80 (production), MOS 6502, C99, QBE (AMD64/ARM64/RISC-V)

1.1 The Compilation Pipeline

```
source.nanz
  |
  ▼ nanz.Parse()
*hir.Module          ← High-level IR: structured control flow,
named vars
  |
  ▼ hir.LowerModule()
*mir2.Module (raw)   ← SSA-like virtual registers, typed ops
  |
  ▼ Optimization passes
*mir2.Module (opt)   ← Constants folded, dead stores removed,
LUTs generated,
                      branches eliminated, conditional
returns sunk
  ▼ Compile-time assertions (MIR2 VM)
  |                               ← Each assert fn(args)==expected runs on
the MIR2 VM
  ▼ Contract optimization (PFCCO)
  |                               ← Interprocedural calling convention
selection
  ▼ PreallocCoalesce
  |                               ← Block-param → block-arg register
unification
  ▼ VIR: Z3 joint isel+regalloc (default)
```

```

|
register allocation      ← SMT solver: instruction selection +
|                        in one pass. Provably optimal for leaf
functions.
|                        PBQP fallback for HasAsm / complex
functions.
  ▼ Peephole optimization (16 rules)
  |                        ← LD r,r elimination, tail call
CALL+RET→JP, etc.
  ▼ Z80Codegen / VIR emit
source.a80              ← MZA-compatible Z80 assembly text
|
  ▼ Compile-time assertions (Z80 binary)
  |                        ← Same asserts now run on the real
assembled binary
  ▼ mza (MZA assembler)
source.bin / .tap       ← Ready to run on Z80 hardware or emulator

```

The compiler runs assertions **twice**: once on the abstract MIR2 VM (fast, catches algorithm bugs) and once on the assembled Z80 binary (catches codegen bugs). If both pass, the function is correct by construction.

1.2 Nanz vs. MinZ

MinZ (.minz) is the original frontend, targeting MIR1 + an older codegen. That pipeline is **frozen** — it works but is not developed further.

Nanz is the **replacement**: same syntax spirit, radically better backend. New programs go in Nanz.

Features only in Nanz: PFCCO contracts, PreallocCoalesce, dual-VM asserts, as cast, mzn native backend, signed comparison, trivial inliner, ForEachEdge, LUTGen, BranchEquiv, CondRetSink+CmpSubCarry, @smc parameters, 6502 backend.

Features only in MinZ (frozen): @error propagation, @define macros, @if/@elif conditional compilation. See [Feature Gap Analysis](#) for the parity roadmap.

Features ported to Nanz in v5: Enums (Chapter 16), type aliases (Chapter 16), module imports (Chapter 17), three string types with interpolation (Chapter 18), pipe/trans named pipelines (Chapter 19).

1.3 Nanz vs. PL/M-80

PL/M-80 (.plm) is an Intel language from the 1970s, used to write CP/M and early microcomputer software. The MinZ compiler includes a complete PL/M-80 parser that compiles PL/M programs through the **same HIR→MIR2→Z80 pipeline** as Nanz.

This means: - 26/26 Intel PL/M-80 Tools reference files parse successfully (100%) - 1338 functions, 943 globals, 11661 statements → HIR → Z80 - PL/M programs benefit from PBQP allocation, LUTGen, and all MIR2 passes

1.4 Design Philosophy

No runtime system. No garbage collector. No dynamic dispatch vtables. Every abstraction is transparent at compile time: lambdas become inline code, interfaces become direct function calls, iterators become DJNZ loops.

Provable by construction. Compile-time assertions checked on two independent VMs catch two classes of bugs that historically slip through: - *Algorithm bugs*: both VMs produce the wrong answer - *Codegen bugs*: the MIR2 VM gives the right answer; the Z80 binary diverges

Target-honest. The optimizer knows it is targeting Z80. It knows that SUB B followed by RET NC is shorter than a branch. It uses the Z80 carry flag to communicate comparison results. It emits DJNZ instead of DEC B / JR NZ. These are not peepholes applied after the fact — they emerge from the MIR2 pass structure.

Multi-target. The same Nanz source can compile to Z80 assembly (production), MOS 6502 assembly (retro), C99 (verification), and QBE IL (native AMD64/ARM64). All backends consume the same MIR2 IR.

Chapter 2: Syntax Reference

2.1 Module Structure

A Nanz source file is a **module**: a flat sequence of top-level declarations that may appear in any order. No imports, no forward declarations required.

```
// Declarations may appear in any order.
struct Vec2 { x: u8, y: u8 }

global origin: Vec2

fun Vec2.add(self: ^Vec2, other: Vec2) -> Vec2 { ... }

interface Shape { area }

fun area(s: Shape) -> u16 { ... }

assert area_of_unit_square() == 1

Comments: // line, /* */ block.
```

2.2 Functions

```
fun name(param1: Type1, param2: Type2) -> ReturnType {
    // body
}
```

fn is accepted as an alias for fun. Functions with no return value use void or omit the -> clause entirely:

```
fun clear(buf: ^u8, n: u8) {
    var i: u8 = 0
    while i < n {
        buf[i] = 0
        i = i + 1
    }
}
```

Multiple return values:

```
fun swap(a: u8, b: u8) -> (u8, u8) {
    return (b, a)
}

fun divmod(a: u8, b: u8) -> (u8, u8) {
    return (a / b, a % b)
}
```

```
// Call site:
let (q, r) = divmod(10, 3)    // q=3, r=1
let (_, r2) = divmod(10, 3)  // discard quotient with _
```

Return values are assigned to registers: pos0→HL (or A for u8), pos1→DE, pos2→B. The _ blank identifier triggers dead store elimination — the discarded value is never computed.

Operator overloading — operators are functions with the operator symbol as name:

```
fun +(a: Vec2, b: Vec2) -> Vec2 {
    return Vec2{ x: a.x + b.x, y: a.y + b.y }
}
// Now: a + b → op_add(a, b) → Vec2_add-style call
```

Struct methods — namespaced with `Type.method`:

```
fun Vec2.scale(self: ^Vec2, factor: u8) -> Vec2 {
    return Vec2{ x: self.x * factor, y: self.y * factor }
}
// v.scale(3) → Vec2_scale(&v, 3) – UFCS, zero cost
```

2.3 Variables

```
var i: u8 = 0          // explicit type, optional initializer
let x = 42             // type inferred (u8)
let y: u16 = 1000      // explicit type overrides inference
```

Use-before-init warning: The compiler tracks uninitialized variables at parse time and emits warnings on use:

```
var ptr: ^u8           // no initializer
let v = ptr^           // ⚠ warning: ptr used before initialization
```

This catches a whole class of bugs that would be silent in C.

2.4 Global Variables

```
global counter: u8 = 0
global screen: [u8; 6912] at(0x4000) // hardware-mapped ZX
Spectrum VRAM
global palette: Color at(0xFF00)      // peripheral mapped at
fixed address
```

The `at(addr)` clause maps the global to a specific Z80 address — no pointer arithmetic needed.

2.5 Control Flow

```
// if / else
if x > 0 {
    do_positive(x)
} else {
    do_non_positive()
}
```

```

// while
while i < n {
    process(arr[i])
    i = i + 1
}

// for i in range
for i in 0..n {
    process(i)
}

// for each element in array
for x in buf[0..n] {
    process(x)
}

// switch
switch state {
    case 0: idle()
    case 1: run()
    default: error()
}

// break / continue
while true {
    if done { break }
    if skip { continue }
    work()
}

```

2.6 Pointers and Arrays

```

var p: ^u8           // typed pointer to u8
var q: ptr           // untyped pointer

let val = p^         // dereference → u8
let elem = p[3]       // index (equivalent to (p+3)^) → u8
p[0] = 42             // store through pointer

let addr = &my_global // address-of

```

Pointer arithmetic is done at the Z80 level via `LD HL, ADD HL, DE`. The programmer does not write offset calculations manually — the compiler emits them from `ptr[i]`.

2.7 Structs

```

struct Color {
    r: u8
    g: u8

```

```

    b: u8
}

struct Vec3d {
    x: u16
    y: u16
    z: u8      // z.offset = 4 (computed at parse time from field
layout)
}

global sky: Color

fun set_sky(r: u8, g: u8, b: u8) {
    sky = Color{ r: r, g: g, b: b }
}

```

Field offsets are computed at parse time from the struct declaration. Mixed-width structs (u8 + u16) lay out correctly with byte-accurate offsets.

Consecutive field stores are fused into an HL-chain:

LD HL, &sky / LD (HL), r / INC HL / LD (HL), g / INC HL / LD (HL), b — 53T vs. 61T for three separate absolute stores. This optimization fires automatically when fields are stored in declaration order.

2.8 Lambdas

```

// Inline lambda expression
let double = |x: u8| { return x * 2 }

// Lambda used in iterator chain (fused – no CALL emitted)
arr.map(|x: u8| x * 2).forEach(|x: u8| process(x), n)

// Lambda capturing outer variable (zero-cost – threaded as block
param)
var sum: u8 = 0
arr.forEach(|x: u8| { sum = sum + x }, n)
// sum is threaded through the DJNZ loop as a register – no heap,
no spill

```

2.9 Casts

Nanz supports two equivalent cast syntaxes:

```

// Function-style cast (original syntax)
let byte = u8(some_u16)      // truncate: take low byte
let word = u16(some_u8)     // zero-extend to 16 bits
let signed = i8(some_u8)    // reinterpret (same bits, signed
semantics)

```

```
// "as" cast (added in v4)
let byte = some_u16 as u8    // same as u8(some_u16)
let word = some_u8 as u16    // same as u16(some_u8)
let signed = some_u8 as i8    // same as i8(some_u8)
```

Both forms produce the same HIR CastExpr and generate identical code. Use whichever reads better in context — as is cleaner in chains:

```
(a + b) as u16 * 256.
```

2.10 Extern Functions

```
@extern fun rom_print(s: ptr) -> void          // resolved at
link time
@extern(0x0010) fun rst_10h(a: u8) -> void      // RST 0x10
(single byte CALL)
@extern(0xBB00) fun bc_sendchar(c: u8) -> void  // CALL 0xBB00
(CP/M BDOS-style)
```

@extern(addr) functions with addr that is a multiple of 8 and $\leq 0x38$ emit RST n (1 byte, 11T). All other @extern(addr) functions emit CALL addr (3 bytes, 17T). The compiler selects the cheaper form automatically.

2.11 Register Annotations on Parameters

```
fun fast_op(@z80_a x: u8, @z80_b count: u8, @z80_hl ptr: ^u8) ->
u8 { ... }
```

Available annotations: @z80_a, @z80_b, @z80_c, @z80_hl, @z80_de. These override the PBQP allocator's choice for that parameter — use only when calling from hand-written assembly that has specific register constraints.

2.12 Ranged Types

```
fun double_angle(a: u8<0..359>) -> u16<0..718> { return u16(a) *
2 }
```

u8<lo..hi> declares a parameter or return with a guaranteed value range. The compiler uses this to: 1. Verify the range at call sites (static check, no runtime overhead) 2. Auto-generate a **lookup table** when the range is small enough (≤ 256 values, pure function)

LUT generation example:

```

fun sin_table(angle: u8<0..255>) -> u8 { ... }
// → Evaluates sin_table(0..255) at compile time via MIR2 VM
// → Emits: sin_table_lut: DB 0, 1, 3, 6, 9, ... (256 bytes)
// → Function body replaced by table lookup: LD HL,
sin_table_lut / LD D,0 / LD E,angle / ADD HL,DE / LD A,(HL) / RET
// → Runtime cost: 6 instructions, ~39T – no computation at all

```

2.13 Interfaces and UFCS

```

interface Animal {
    speak
    eat
}

fun Dog.speak(self: Dog) { ... }
fun Cat.speak(self: Cat) { ... }

// Interface as parameter type – monomorphized at compile time:
fun feed(a: Animal) {
    a.speak()    // → Dog_speak(a) or Cat_speak(a) depending on
concrete type
}

```

Cost: zero. No vtable. No fat pointer. The concrete type is resolved at compile time. `a.speak()` emits a direct `CALL Dog_speak` or `CALL Cat_speak`.

UFCS — uniform function call syntax — lets you write `a.method(args)` for any function `fun Type.method(self: Type, args)`. It desugars at parse time:

```
v.scale(3)    →  Vec2_scale(&v, 3)    →  CALL Vec2_scale
```

2.14 Inline Assembly

```

fun fast_clear() {
    asm {
        XOR A
        LD (HL), A
        INC HL
        DJNZ -3
    }
}

// Target-gated: only included for Z80 backend
fun platform_init() {
    asm z80 {
        DI
        LD SP, 0xFFFF
        EI
    }
}

```

```

    }
}

```

Inline assembly blocks emit Z80 instructions verbatim. The `asm z80` variant is only included when compiling for Z80.

Full Syntax

```
asm TARGET? (in REG, ...)? (ret REG)? (clob REG,... | auto | all)?
{ ... }
```

Clause	Takes	Default	Purpose
(in REG,...)	Physical registers	Auto-infer from @z80_* params	Liveness: keep these registers alive
(ret REG)	Physical register	void	Return value register
(out REG)	—	—	Alias for ret
(clob REG,... auto all)	Physical registers or keyword	auto	Clobber specification

All three clauses use physical register names (A, B, C, HL, DE, etc.) for consistency.

Returning Values: (ret REG)

Use (ret REG) to declare which register holds the asm block's return value:

```
fun zx_peek(@z80_hl addr: u16) -> u8 {
    asm z80 (ret A) { LD A, (HL) }
}
// Z80 output: LD A, (HL) / RET - 2 instructions

fun double(@z80_a x: u8) -> u8 {
    asm z80 (ret A) (clob A, F) { ADD A, A }
}
// Z80 output: ADD A, A / RET - 2 instructions
```

Without (ret), the asm block is void. If a function ends with an asm block and has no explicit return, the compiler uses implicit return (the value already in the return register).

Clobber Specification: (clob ...)

The clob clause tells the register allocator which registers the asm block destroys:

```
// Explicit clobber list:
asm z80 (ret A) (clob A, F) { ADD A, A }

// Auto-detect (default): compiler parses asm text
asm z80 (ret A) { LD A, (HL) }
// Compiler sees: LD writes A, flags always touched → clob {A, F}

// Escape hatch for opaque code:
asm z80 (clob all) { CALL unknown_routine }
```

When no clob clause is given, the compiler auto-analyzes the asm text:

- Extracts destination registers from known instructions (LD, ADD, INC, etc.)
- Always includes F (flags) — almost every Z80 instruction touches flags
- CALL/RST or unknown mnemonics → falls back to clob all

This is a major improvement over the old behavior (which always assumed all registers clobbered, causing excessive spills on Z80's limited register file).

Input Operands: (in REG)

Use (in REG) to declare which registers the asm block reads. This is an optimization hint — when omitted, the compiler auto-infers from @z80_* annotated parameters:

```
// Without (in) – auto-inferred from @z80_hl and @z80_a:
fun zx_poke(@z80_hl addr: u16, @z80_a val: u8) {
  asm z80 { LD (HL), A }
}

// With (in) – explicit, only A is marked live:
fun foo(@z80_a x: u8, @z80_hl y: u16) -> u8 {
  asm z80 (in A) (ret A) { ADD A, 42 }
  // HL is free for the allocator – not marked live through asm
}
```

For backward compatibility, variable names also work: (in addr) is resolved via the old path.

Complete I/O Example

```
// Memory read/write
fun zx_peek(@z80_hl addr: u16) -> u8 {
  asm z80 (ret A) { LD A, (HL) }
```

```

}

fun zx_poke(@z80_hl addr: u16, @z80_a val: u8) {
    asm z80 { LD (HL), A }
}

// Keyboard: read row via port 0xFE
fun zx_key_row(@z80_a port: u8) -> u8 {
    asm z80 (ret A) { IN A, (0xFE) }
}

// Console output (emulator stdout port)
fun console_log(@z80_a n: u8) {
    asm z80 { OUT (0x23), A }
}

```

Each function compiles to exactly 2 instructions (operation + RET).

2.15 ptr() — Direct Memory Access

The `ptr(expr)` cast converts a `u16` address to a pointer, enabling direct memory read/write without inline asm:

```

// Read byte at address (peek):
let val: u8 = ptr(0x5800)^

// Write byte to address (poke):
ptr(0x5800)^ = 0x38

// As functions:
fun peek(addr: u16) -> u8 { return ptr(addr)^ }
fun poke(addr: u16, val: u8) { ptr(addr)^ = val }

```

Generated Z80:

```

peek:
    LD A, (HL)      ; 2 instructions – zero overhead
    RET
poke:
    LD (HL), C      ; 2 instructions
    RET

```

`ptr()` is consistent with other cast constructors (`u8()`, `u16()`, `i8()`). The cast is a no-op at the machine level (`u16` and `ptr` are both 16-bit), and the `deref ^` produces a standard load or store.

This eliminates the need for asm wrappers for memory-mapped I/O — pure language, zero overhead.

2.16 |> Value Pipe Operator

The |> operator chains function calls, inserting the left-hand expression as the first argument:

```
expr |> f           // → f(expr)
expr |> f(a, b)     // → f(expr, a, b)
```

Example:

```
fun double(x: u8) -> u8 { return (x + x) }
fun inc(x: u8) -> u8 { return (x + 1) }

fun piped() -> u8 {
    return 5 |> double |> inc    // = inc(double(5)) = 11
}
```

Generated Z80:

```
piped:
    LD A, 11    ; constant-folded at compile time!
    RET
```

The entire chain is evaluated at compile time when inputs are constants. For runtime values, each |> compiles to a normal function call with no overhead.

2.17 sizeof(Type)

sizeof(Type) is a compile-time constant expression that evaluates to the size in bytes of a type. It is resolved at parse time via resolveTypeSize().

```
sizeof(u8)      // → 1
sizeof(u16)     // → 2
sizeof(bool)    // → 1
sizeof(i16)     // → 2
sizeof(u32)     // → 4
```

For user-defined structs, sizeof computes the total layout from field widths:

```
struct Arena {
    ptr: u16    // 2 bytes
    end: u16    // 2 bytes
}
```

```
sizeof(Arena)   // → 4
```

```
struct Sprite {
```

```

    x: u8      // 1 byte
    y: u8      // 1 byte
    frame: u8  // 1 byte
    tile: u8   // 1 byte
}

```

```
sizeof(Sprite) // → 4
```

```

struct Vec3d {
    x: u16      // 2 bytes
    y: u16      // 2 bytes
    z: u8       // 1 byte
}

```

```
sizeof(Vec3d) // → 5
```

`sizeof` is a first-class expression — it can appear anywhere a constant integer is valid:

```

// Typed allocation
let enemy_ptr = arena.alloc(sizeof(Enemy))

```

```

// Array stride calculation
let offset = index * sizeof(Entry)

```

```

// Compile-time assertion
assert sizeof(Color) == 3

```

Because it resolves at parse time, `sizeof` has zero runtime cost. The compiler substitutes the integer literal directly into the generated code.

Chapter 3: Type System

3.1 Numeric Types

Type	Width	Description
u8	8-bit	Unsigned byte — Z80 registers A/B/C/D/E/H/L
u16	16-bit	Unsigned word — Z80 register pairs HL/DE/BC
i8	8-bit	Signed byte (same registers, signed arithmetic)
i16	16-bit	Signed word
u24	24-bit	24-bit unsigned (eZ80 / Agon Light 2 native)
i24	24-bit	24-bit signed (eZ80 native, MZV)
u32	32-bit	32-bit via Z80 EXX shadow pair (HL ' /DE ' /BC ')
i32	32-bit	Signed 32-bit (MZV VM, shadow pair on Z80)

Type	Width	Description
f8.8	16-bit	Fixed-point: 8 integer bits + 8 fractional bits
f16.8	24-bit	Fixed-point: 16 integer + 8 fractional
f8.16	24-bit	Fixed-point: 8 integer + 16 fractional
f16.16	32-bit	Fixed-point: 16 integer + 16 fractional
bool	8-bit	false=0, true≠0
void	—	Return type only

Fixed-point types parse with dot notation: f8.8 is “f” followed by “8” (integer bits) “.” “8” (fractional bits). The compiler handles arithmetic (add/sub exact, mul/div require shifts that the optimizer emits).

3.2 Signed Arithmetic

Signed types (i8, i16) use the same physical registers as unsigned but with signed comparison semantics:

```
fun max_i8(a: i8, b: i8) -> i8 {
    if a > b { return a }
    return b
}

assert max_i8(5, 3) == 5           // both positive
assert max_i8(251, 5) == 5        // 251 = -5 in i8, so max(-5, 5)
= 5
assert max_i8(251, 253) == 253    // max(-5, -3) = -3 = 253
```

How it works: Values are stored truncated — i8(-5) is 251 in memory. The lowerer picks CmpGe for signed and CmpUge for unsigned (via IsSigned()). The MIR2 VM sign-extends both operands before signed comparison: signExtend(251, 8) = -5. This makes (-5) < 5 evaluate to true.

On Z80 hardware, signed comparison uses the S^V flag combination (sign XOR overflow) — the same approach the Z80 was designed for.

3.3 Pointer Types

Type	Description
ptr	Untyped 16-bit Z80 address
^T	

Type	Description
	Typed pointer to T (human-readable; same as ptr at machine level)

^Struct pointer receivers enable clean method syntax:

```
fun Acc.add(self: ^Acc, amount: u8) -> u8 {
    self.val = self.val + amount
    return self.val
}
// self^.val also works (explicit dereference)
```

3.4 The u32 “ClassDWord” via EXX Shadow Pair

32-bit values on Z80 — without a 32-bit bus — use the **EXX shadow register pair** trick:

```
fun add32(a: u32, b: u32) -> u32 { return a + b }
```

Generated Z80:

```
; fun add32(a: u32 = HL/DE', b: u32 = HL'/DE) -> u32 = HL/DE
add32:
    ADD HL, DE      ; low 16 bits: HL += DE
    EXX             ; swap to shadow pair
    ADC HL, DE      ; high 16 bits: HL' += DE' + carry
    EXX
    RET
```

5 instructions. The PBQP allocator places the 32-bit halves in the main and shadow HL — they don’t interfere with each other because EXX separates them.

3.5 Calling Convention

The calling convention is **not fixed** — it is computed per function by the PBQP register allocator and interprocedural contract optimizer (PFCCO). Typical Z80 mapping:

Class	Z80 register	Typical use
ClassAcc	A	First u8 param, return value
ClassCounter	B	Second u8 param, loop counter
ClassPointer	HL	u16 params, pointer args, return value
ClassIndex	DE	Second u16 param
ClassPair	BC	Third param or general pair

Class	Z80 register	Typical use
ClassGeneral	C/D/E/H/L	Remaining 8-bit params
ClassDWord	HL+shadow	u32 values

The **contract optimizer** (PFCCO) searches across the call graph for the assignment that minimizes total T-states for caller+callee. The result is that calling conventions are tailored to the specific set of functions in your program — not a fixed ABI.

Example: If function $f(a, b)$ always calls $g(b)$, PFCCO will assign b to the same register in both functions, eliminating the move at the call site. This is computed globally, not locally — the optimizer sees the entire call graph.

Chapter 4: Structs, Methods, and Interfaces

4.1 Struct Declaration and Layout

```
struct Sprite {
    x: u8      // offset 0
    y: u8      // offset 1
    frame: u8   // offset 2
    tile: u8    // offset 3
}
```

The compiler computes byte offsets at parse time: x at 0, y at 1, $frame$ at 2, $tile$ at 3. Mixed-width structs with $u16$ fields get 2-byte offsets:

```
struct Vec3d {
    x: u16    // offset 0
    y: u16    // offset 2
    z: u8     // offset 4
}
```

4.2 Methods and UFCS

```
fun Sprite.move(self: ^Sprite, dx: i8, dy: i8) {
    self.x = u8(i8(self.x) + dx)
    self.y = u8(i8(self.y) + dy)
}
```

```
// Call site:
sprite.move(+1, 0)  →  Sprite_move(&sprite, 1, 0)  →  CALL
Sprite_move
```

Zero overhead. The method table exists only in the parser — no runtime representation.

4.3 Interfaces and Static Dispatch

```
interface Drawable {
    draw
}

fun Circle.draw(self: ^Circle) { ... }
fun Rect.draw(self: ^Rect) { ... }

fun render_all(shape: Drawable) {
    shape.draw()
}

// render_all(my_circle):
//   → only one implementation of 'draw' for Circle exists
//   → monomorphized to: CALL Circle_draw
```

When multiple implementors exist, the compiler requires a statically known concrete type at the call site. When a function parameter is typed as an interface and only one implementor exists in the module, the call is **monomorphized automatically** — no code change required.

4.4 impl Blocks (v7.0)

Group methods by trait and type — desugars to UFCS functions:

```
struct Circle { x: u8, y: u8, radius: u8 }
struct Rect { x: u8, y: u8, w: u8, h: u8 }

interface Shape { area, perimeter }

impl Shape for Circle {
    fun area(self) -> u8 {
        return 3 * self.radius * self.radius
    }
    fun perimeter(self) -> u8 {
        return 6 * self.radius
    }
}

impl Shape for Rect {
    fun area(self) -> u8 { return self.w * self.h }
    fun perimeter(self) -> u8 { return 2 * self.w + 2 * self.h }
}
```



```
// Usage – UFCS dispatch:
var c: Circle
c.radius = 5
c.area()          // calls Circle_area(&c) → 75
c.perimeter()     // calls Circle_perimeter(&c) → 30
```

Desugaring: `impl Shape for Circle { fun area(self) -> u8 { ... } }` becomes `fun Circle_area(self: ^Circle) -> u8 { ... }`. The `self` parameter is automatically typed as `^TypeName`. Methods with extra parameters work naturally:

```
impl Ops for Counter {
    fun add(self, n: u8) -> u8 { return self.val + n }
}
c.add(5) // calls Counter_add(&c, 5)
```

Zero runtime overhead. The `impl` block is pure syntax sugar — no vtables, no indirection. Everything resolves at compile time to direct CALL instructions.

4.5 Operator Overloading

```
struct Vec2 { x: u8, y: u8 }

fun +(a: Vec2, b: Vec2) -> Vec2 {
    return Vec2{ x: a.x + b.x, y: a.y + b.y }
}

let v1 = Vec2{ x: 10, y: 20 }
let v2 = Vec2{ x: 5, y: 3 }
let v3 = v1 + v2 // → op_add(v1, v2) → Vec2_add-like dispatch
```

Operators for primitive types (`u8 + u8`) use Z80 ALU instructions directly — overloading only fires when one operand is a struct type.

Vec3 with operator overloading — a 3D wireframe building block:

```
struct Vec3 { x: i8, y: i8, z: i8 }

fun +(a: Vec3, b: Vec3) -> Vec3 {
    return Vec3 { x: a.x + b.x, y: a.y + b.y, z: a.z + b.z }
}

fun midpoint(a: Vec3, b: Vec3) -> Vec3 {
    return Vec3 {
        x: (a.x + b.x) >> 1, // division by 2 via arithmetic
        y: (a.y + b.y) >> 1,
        z: (a.z + b.z) >> 1
    }
}
```

```
}  
}
```

Zero-cost: the compiler inlines structs into registers. PFCCO picks the optimal layout ($x \rightarrow H$, $y \rightarrow L$, $z \rightarrow D$ or whatever minimizes total moves).

Chapter 5: Iterator Chains

Nanz supports a composable iterator chain syntax on arrays and pointers. The crucial property: **the chain is fused at compile time into a single DJNZ loop**. No intermediate arrays. No function pointer overhead. No virtual dispatch.

5.1 Combinators

Method	Meaning
<code>ptr.map(λ)</code>	Transform each element
<code>ptr.filter(λ)</code>	Keep elements where λ is true
<code>ptr.forEach(λ, n)</code>	Execute λ for each of n elements
<code>ptr.fold(init, λ)</code>	Reduce n elements to a single value
<code>ptr.reduce(λ)</code>	Reduce with first element as init
<code>ptr.take(k)</code>	Keep first k elements
<code>ptr.skip(k)</code>	Skip first k elements
<code>ptr.enumerate()</code>	Add element index
<code>ptr.chain(other)</code>	Concatenate two iterators

5.2 Fusion

```
arr.map(|x: u8| x * 2).filter(|x: u8| x > 10).forEach(|x: u8|  
process(x), n)
```

The parser recognizes this chain pattern. The HIR lowerer's `recognizeIterChain` function fuses all stages before emitting any MIR2 instructions. The result is a single loop:

```
.loop:  
    LD A, (HL)           ; load element  
    INC HL               ; advance pointer  
    ADD A, A             ; map: x * 2 (strength-reduced from MUL)  
    CP 11                ; filter: x > 10 → x >= 11  
    JR C, .skip          ; skip if filter fails
```

```

CALL process      ; forEach body
.skip:
DJNZ .loop        ; B--; branch if B ≠ 0

```

No intermediate storage. No lambda calls. The filter check and map transform are inlined.

5.3 Closure Capture

Lambdas inside chains can capture and mutate outer variables:

```

var sum: u8 = 0
arr.forEach(|x: u8| { sum = sum + x }, n)

```

sum is not spilled to memory. It is threaded through the DJNZ loop as a **block parameter** — a loop-carried SSA value that lives in a register:

```

LD B, n          ; counter
LD C, 0          ; sum = 0 (in C)
.loop:
LD A, (HL)       ; x
ADD A, C         ; sum + x
LD C, A          ; sum = result
INC HL
DJNZ .loop
LD A, C          ; move sum to A for return/use

```

This is **zero-cost closure capture** — sum is a CPU register throughout the loop.

5.4 E2E Verification

All 11 iterator chain combinations are verified end-to-end:

Chain	Binary	T-states
forEach	hex-verified	~43T/elem
map+forEach	hex-verified	~50T/elem
filter+forEach	hex-verified	~52T/elem
map+filter+forEach	hex-verified	~57T/elem
take+forEach	hex-verified	~43T/elem
skip+forEach	hex-verified	~43T/elem
lambda map	hex-verified	~50T/elem
lambda filter	hex-verified	~52T/elem
multi-stage	hex-verified	varies

Chain	Binary	T-states
fold	hex-verified	~30T/elem
forEach with capture	hex-verified	~30T/elem

“Hex-verified” means: compiled to Z80 binary, loaded into the MZE emulator, executed, result checked against expected value.

Chapter 6: range(lo..hi) — Counter-Based Iteration

6.1 Overview

range(lo..hi) is a **counter-based iterator source** — no memory pointer, no LD A, (HL), no INC HL. Elements are the DJNZ counter value itself, counting down from hi-lo to 1.

```
fun sum_range(n: u8) -> u8 {
    return range(0..n).fold(0, |acc: u8, i: u8| { return acc +
i })
}
```

Generated Z80:

```
; fun sum_range(n: u8 = A) -> u8 = A
sum_range:
    LD B, 0                ; acc init = 0
    AND A                  ; pre-check: n == 0?
    JRS NZ, .trmp0
    JRS .rng_exit2
.rng_body1:
    ADD A, B                ; acc += counter ← ONE instruction per
iteration
    DJNZ .rng_body1        ; B--; branch if B ≠ 0
    LD B, A
    JP .rng_exit2
.rng_exit2:
    LD A, B
    RET
.trmp0:
    LD C, A                ; save n (parallel copy resolver)
    LD A, B                ; A = 0 (acc init)
    LD B, C                ; B = n (counter)
    JRS .rng_body1
```

Body: one instruction per iteration — **ADD A, B**. No loads. No stores. No spills.

6.2 Counting Semantics

`range(0..n)` counts **DOWN**: DJNZ starts at $B=n$ and decrements to 0. The element values are $n, n-1, \dots, 1$ — the counter itself.

This means `range(0..n).fold(0, |acc,i| acc+i)` computes `**sum(1..n) = n*(n+1)/2**`, not `sum(0..n-1)`. This is intentional: counting down is the natural direction for DJNZ, and computing the triangular number is the correct mathematical result.

n	Expected result	Formula
0	0	$n=0$: loop doesn't run
1	1	just 1
4	10	$4+3+2+1$
5	15	$5+4+3+2+1$
10	55	$10 \times 11 / 2$

All five verified on the Z80 binary via `TestRangeFold_E2E_SumRange`.

6.3 The Parallel Copy Bug: Caught by Dual-VM

When the range fold first compiled with correct semantics, the Z80 binary produced wrong results even though the MIR2 VM gave correct answers. This is exactly the class of **codegen bug** that would be invisible without dual-VM verification.

Root cause: The parallel copy resolver used register A as a scratch for cycle-breaking. For the $A \leftarrow B$ cycle in the range fold trampoline, both A and B ended up as 0. The counter never loaded n.

Fix: Collect the full set of registers in the cycle (`cycleRegs`). If A is in the cycle, pick the first non-cycle 8-bit register as scratch:

```
; Correct parallel copy for A ← B cycle, scratch=C:
LD C, A      ; save n (A's original value)
LD A, B      ; A = 0 (acc init)
LD B, C      ; B = n (counter)
```

This bug was invisible to the MIR2 VM. The dual-VM assertion system caught it immediately.

Chapter 7: Compile-Time Assertions and Sandbox Blocks

7.1 The Idea

Compile-time assertions are checked as part of compilation. If an assertion fails, the build fails. No separate test runner needed.

```
fun gcd(a: u8, b: u8) -> u8 {  
    while b != 0 {  
        let t = b  
        b = a % b  
        a = t  
    }  
    return a  
}
```

```
assert gcd(12, 8) == 4  
assert gcd(100, 75) == 25  
assert gcd(17, 13) == 1
```

These three assertions run every time the code compiles. If you break gcd, you know immediately.

7.2 Dual-VM Execution

By default, **each assertion runs twice**:

1. **MIR2 VM** — fast abstract interpreter. Runs on the SSA-form intermediate representation, before register allocation. Catches algorithm bugs.
2. **Z80 binary** — assembles the generated asm, loads into MZE (the Z80 emulator), calls the function, reads the result register. Catches codegen bugs.

```
[MIR2 optimization complete]  
↓  
assert gcd(12, 8) == 4 ← MIR2 VM: fast, ABI-agnostic  
↓  
[PBQP register allocation + Z80 codegen]  
↓  
assert gcd(12, 8) == 4 ← Z80 binary: slow, bit-exact
```

If both pass: the function is correct by construction.

7.3 What Each VM Catches

Situation	MIR2 VM	Z80 binary	What happened
sum_range(5) = 15	15	15	All good
sum_range(5) = 7	7	7	Algorithm bug
sum_range(5) = 15	15	0	Codegen bug (e.g. the A↔B swap)

The third row is the canonical example. The MIR2 VM is ABI-agnostic — it doesn't simulate physical registers, so it cannot observe the swap corruption. The Z80 binary check catches it immediately.

7.4 Syntax Variants

```
// Default: run on both MIR2 VM and Z80 binary
assert sum_range(5) == 15

// Only run on MIR2 VM (fast – skips Z80 assembly + emulator)
assert sum_range(5) == 15 via mir2

// Only run on Z80 binary (skips MIR2 VM)
assert sum_range(5) == 15 via z80
```

7.5 Multi-Value Assertions

```
fun divmod(a: u8, b: u8) -> (u8, u8) { ... }

assert divmod(10, 3) == (3, 1)    // quotient=3, remainder=1
```

Multi-return functions return a tuple. The assert syntax compares each return value independently.

7.6 How Z80 Asserts Work

The Z80 assert runner uses the **actual register allocation** to build the calling bootstrap:

1. Look up the compiled function's `Contract.Params` from the MIR2 module
2. Look up each param's physical location from the `AllocResult` (the output of PBQP)

3. Emit LD <actual_reg>, arg_value — correct even if the optimizer chose C instead of B
4. Emit CALL funcname + DI / HALT
5. Assemble with MZA, load into MZE, run
6. Read the result from the return register (determined by Contract>Returns[0].Class)

This means Z80 asserts are **robust to calling convention changes**: if the contract optimizer re-assigns params to different registers between compiler versions, the assert runner automatically adapts.

7.7 Sandbox Blocks: Shared VM State

Top-level assert statements each get a **fresh VM instance**. This provides isolation — one assert cannot observe the side effects of another. But sometimes you *want* shared state: testing a sequence of operations that build on each other.

sandbox blocks group assertions that share a single VM instance:

```
sandbox "arena lifecycle" {  
    assert arena_init(0xC000, 256) == 0 via mir2  
    assert arena_alloc(4) == 0xC000 via mir2  
    assert arena_alloc(4) == 0xC004 via mir2  
    assert arena_remaining() == 248 via mir2  
    assert arena_reset(0xC000) == 0 via mir2  
    assert arena_remaining() == 256 via mir2  
}
```

Semantics:

- **Fresh VM (top-level assert):** Each assert creates a new VM, calls the function, checks the result, discards the VM. Global variables start at zero. No side effects survive between assertions.
- **Shared VM (sandbox):** One VM is created for the entire sandbox block. Assertions execute in order on the same VM. Mutations to globals (like an arena's bump pointer) persist across assertions within the sandbox.

This distinction matters for any code that maintains state in globals — arena allocators, counters, state machines, initialization sequences.

Use cases:

1. **Sequential mutations:** Verify that alloc advances a pointer, then reset restores it.

2. **Cross-function state sharing:** Test that `init + alloc + remaining` all see the same arena.
3. **Cumulative effects:** Assert that repeated calls accumulate correctly (e.g. summing into a global).

Both backends support sandboxes. The MIR2 VM simply reuses the same VMState across assertions. The Z80 backend uses a fixed-size 64-byte NOP-padded trampoline to ensure stable addresses across re-assemblies — each assert in the sandbox is assembled into the same trampoline slot, and `Unhalt()` resumes the emulator between assertions rather than resetting it. This guarantees that global memory (the arena state, counters, etc.) is preserved across Z80 sandbox assertions just as it is on the MIR2 VM.

Sandbox vs. top-level — choosing the right mode:

Scenario	Use
Pure function (no side effects)	Top-level assert
Stateful sequence (globals mutated)	sandbox block
Mix of both	Top-level for pure, sandbox for stateful

Chapter 8: The Optimization Pipeline

8.1 Overview of Passes

LUTGen	← replace bounded pure functions with lookup tables
PropagateConstants	← find params that are always the same value
FoldConstants	← evaluate constant expressions ($1+2 \rightarrow 3$)
SimplifyIdentities	← $x+0 \rightarrow x$, $x*1 \rightarrow x$, $x-x \rightarrow 0$, etc.
ConstantCallElim	← calls with all-const args $\rightarrow$ folded to result
DeadStoreElim	← remove unused instructions
BranchEquiv	← remove redundant conditional branches (VM-proved)
CondRetSink	← convert <code>BrIf-with-trivial-else</code> to
TermCondRet	
hoistReorder	← move <code>Sub</code> before <code>Cmp</code> for <code>CmpSubCarry</code> fusion
CmpSubCarry	← replace <code>Cmp+Sub</code> pair with single carry-flag result
ContractOpt (PFCC0)	← interprocedural calling convention optimization
PreallocCoalesce	← block-param $\leftrightarrow$ block-arg register unification

PBQPAAllocate	← physical register assignment (cost-weighted)
CopyCoalesce	← eliminate redundant moves across block boundaries
TrivialInliner	← inline single-instruction callees (swap→0 insts)
Z80Codegen	← emit Z80 assembly text

8.2 LUT Generation

Any pure function with a `u8<lo..hi>` ranged parameter where the range fits in ≤ 256 values is replaced with a lookup table at compile time:

```
// Before: 50-instruction sin computation
fun sin(a: u8<0..255>) -> u8 { ... }

// After LUTGen (at compile time):
// sin_lut: DB 0, 1, 3, 6, 9, 12, ... ← 256 bytes, computed via MIR2 VM
// fun sin(a: u8) → LD HL, sin_lut / ADD HL,DE / LD A,(HL) / RET
// (6 insts, ~39T)
```

8.3 BranchEquiv

The BranchEquiv pass uses the MIR2 VM to prove when a conditional branch is redundant:

Example: In the MIR2 IR for `abs_diff`, a `CmpEq` guard appears after optimizations. At the equality boundary `a == b`, both `a - b = 0` and `b - a = 0`. The branch is provably dead. Run the VM with 256 boundary inputs (`v, v`) for all `v`. If both sides return the same value, the branch is dead. Replace `BrIf(CmpEq) → Jmp`. Save 10T + 3 bytes per call.

8.4 CondRetSink and Flag Fusion: `abs_diff` in 4 Instructions

This five-pass optimization transforms `abs_diff` from 8 instructions to 4:

Input:

```
fun abs_diff(a: u8, b: u8) -> u8 {
    if a < b { return b - a }
    return a - b
}
```

Pass 1: CondRetSink → hoists trivial else-block, converts BrIf to TermCondRet (= RET CC) **Pass 2: SubSwapNeg** → $b - a$ when we already have $a - b$ → NEG **Pass 3: hoistReorder** → moves Sub before Cmp so carry flag contains comparison result **Pass 4: CmpSubCarry** → replaces Cmp with no-op (carry already set by SUB) **Pass 5: PBQP** → no interference, both map to A

Final output: 4 instructions.

```
abs_diff:    ; a=A, b=C
SUB C        ; A = a-b, carry = (a < b unsigned)
RET NC       ; if a >= b: return a-b (4T+10T = 14T)
NEG          ; A = -(a-b) = b-a (8T)
RET          ; (10T)
```

8.5 PreallocCoalesce: DJNZ from 5 Instructions to 1

New in v4. Before PBQP allocation, PreallocCoalesce unifies block-parameter virtual registers with their corresponding block-argument virtual registers when live ranges don't overlap.

Before PreallocCoalesce (ex7_mapinplace):

```
LD A, 1
NEG
ADD A, B
LD B, A
JRS .add2_inplace_fe_head1
```

After PreallocCoalesce:

```
DJNZ .add2_inplace_fe_body2
```

Saving: 4 instructions, ~30T per iteration. The counter was unified with register B, allowing the back-edge to emit a single DJNZ instead of manual decrement + jump.

Impact across 6 showcase files: - mapInPlace: 5 instructions → 1 DJNZ - factorial_fold: mul16 routine eliminated entirely - forEach/max_chain: trampoline block removed - fib_iter: 3 EX DE,HL instructions eliminated - fib_fold: 6 redundant register moves removed

8.6 Trivial Inliner

New in v4. When a callee is a trivial function (single instruction or alias), the inliner replaces the call entirely:

```
fun swap(a: u8, b: u8) -> (u8, u8) { return (b, a) }
fun min_of(a: u8, b: u8) -> u8 { return minmax(a, b).0 }
```

- `swap(a,b).1 == a` → **zero instructions** (the compiler proves the identity statically)
- `min_of(a,b)` → EQU `minmax` (**0 bytes** — just an alias label)

8.7 Interprocedural Contract Optimization (PFCCO)

The contract optimizer searches for the best calling convention across the entire call graph:

1. **Build call graph** — topological sort (callees before callers)
2. **Candidate choices** — cartesian product of plausible register classes for each param
3. **Conflict filtering** — reject assignments where two params must share one physical reg
4. **Edge cost** — cost of crossing each call edge for a given contract pair
5. **Greedy DP** — assign contracts to minimize total T-states across all callers

Result: for a function called in a tight loop, the optimizer assigns the param directly to the register the loop already has the value in — eliminating all move instructions at the call site.

8.8 Multiply Strength Reduction

Standard power-of-2: $N \times \text{ADD HL}, \text{HL}$.

Byte-boundary optimization: $* 256$ = “move low byte to high byte”:

```
; x * 256:
LD H, L      ; H = L (low byte becomes high byte)
LD L, 0      ; L = 0
; result: HL = x * 256 in 8T, 2 bytes (was: 56T, 16 bytes for
8xADD HL,HL)
```

For $* 512$, $* 1024$ — byte-swap + remaining shifts. **Small composites** (3, 5, 6, 9): PUSH+POP+shift+add sequences (no loop).

Chapter 9: Multiple Compilation Targets

Nanz/MIR2 supports multiple output backends. The same source compiles to different targets from the same IR.

9.1 The Backend Spectrum

```
*mir2.Module
├── Z80Codegen          → .a80 assembly [production]
├── M6502Codegen        → .s   assembly [retro: Apple II, C64,
BBC Micro]
├── mir2c.Codegen       → .c   file      [verification +
portability]
├── mir2qbe.Codegen     → .ssa (QBE)   [native: x86-64,
ARM64, RISC-V]
└── (planned) mir2llvm → LLVM IR      [future]
```

9.2 MOS 6502 Backend

New in v4. The 6502 backend compiles Nanz through the same MIR2 pipeline to 6502 assembly. **35/35 tests pass** with a dual-VM oracle (MIR2 VM vs sim6502).

```
fun abs_diff(a: u8, b: u8) -> u8 {
  if a < b { return b - a }
  return a - b
}
```

```
; 6502 output:
abs_diff:
  SEC
  SBC param_b      ; A = a - b
  BCS .done        ; if a >= b, done
  EOR #$FF         ; NEG via complement + 1
  ADC #$01
.done:
  RTS
```

Console I/O adapters for Apple II, Commodore 64, and BBC Micro are included for testing.

9.3 MIR2→C: Verification Backend

MIR2→C is a verification and portability tool. It translates MIR2 to C99 that gcc/clang can compile and run:

```
// Generated by mir2c:
uint8_t abs_diff(uint8_t a, uint8_t b) {
  if (a < b) return b - a;
  return a - b;
}
```

Uses: Cross-checking (Z80 vs host), portability target (play-test game logic on a fast machine before deploying to Z80), reference for overflow semantics.

9.4 MIR2→QBE: Native Backend

QBE is a minimalist compiler backend that compiles `.ssa` files to x86-64, ARM64, or RISC-V native code.

```
# Generated QBE for abs_diff
export function w $abs_diff(w %a, w %b) {
@entry
    %cond =w cultw %a, %b
    jnz %cond, @then, @else
@then
    %r1 =w sub %b, %a
    ret %r1
@else
    %r2 =w sub %a, %b
    ret %r2
}
```

9.5 The mzn Native Compiler

New in v4. The `mzn` CLI compiles Nanz directly to native AMD64 executables:

```
# Compile via QBE (default)
mzn program.nanz

# Compile via C99
mzn -c program.nanz

# Both backends
mzn -c -q program.nanz

# Emit C99 source (inspect only)
mzn -emit-c program.nanz

# Emit QBE IL (inspect only)
mzn -emit-qbe program.nanz
```

This enables native-speed testing of Nanz programs on the development machine — the same logic runs on both Z80 and AMD64.

9.6 The Big Picture: Four Verification Targets

```
abs_diff(10, 3) == 7
  checked by:  MIR2 VM (abstract)
               Z80 binary via MZE (physical Z80)
               6502 binary via sim6502 (physical 6502)
               C binary via gcc (host native)
               QBE binary via QBE (modern native)
```

If all five agree, you can be extremely confident the function is correct.

9.7 Z80 Platform Targets

Target flag	Platform	Notes
--target=spectrum	ZX Spectrum 48K/ 128K	Default entry 0x8000, screen at 0x4000
--target=cpm	CP/M systems	TPA entry 0x0100, BDOS at 0x0005
--target=agon	Agon Light 2 (eZ80)	MOS API, VDP graphics, u24 native
--target=generic	Bare Z80	No platform assumptions

Chapter 10: Z80 Extern and Register Contracts

10.1 Basic @extern

```
@extern fun process(x: u8) -> void
```

The compiler assigns register classes to process's parameter as normal. It will probably put x in A (ClassAcc). At the call site: LD A, value / CALL process.

10.2 RST Optimization

```
@extern(0x10) fun rst_16(c: u8) -> void    // RST 0x10
@extern(0x28) fun rst_40(c: u8) -> void    // RST 0x28
@extern(0xBB00) fun bdos_call(c: u8) -> void // CALL 0xBB00
```

The compiler emits: - RST *n* for addresses that are multiples of 8 and $\leq 0x38$ (1 byte, 11T) - CALL *addr* for all other addresses (3 bytes, 17T)

10.3 Annotated @extern: Precise Register Contracts

```
@extern fun LD_BYTES(@z80_a type: u8, @z80_de dest: ptr, @z80_b
count: u8) -> void
// Spectrum ROM 0x0556: A=type, DE=dest, BC=length
```

The @z80_* annotations override PBQP for those parameters.

10.4 The ABI Is Not Fixed

Unlike traditional languages, Nanz does not have a fixed calling convention. You can observe the chosen convention in the generated assembly comment:

```
; fun abs_diff(a: u8 = A, b: u8 = C) -> u8 = A ; clobbers: F
```

Chapter 11: Verified Codegen — Showcase

24 showcase examples, all verified. Here are the highlights.

11.1 abs_diff: Five Passes to Four Instructions

```
fun abs_diff(a: u8, b: u8) -> u8 {
  if a < b { return b - a }
  return a - b
}
```

```
assert abs_diff(10, 3) == 7
assert abs_diff(3, 10) == 7
assert abs_diff(5, 5) == 0
```

```
; fun abs_diff(a: u8 = A, b: u8 = C) -> u8 = A ; clobbers: F
abs_diff:
  SUB C      ; a - b, carry = (a < b)
  RET NC     ; a >= b: return a-b
```



```

NEG          ; -(a-b) = b-a
RET

```

4 instructions. Both MIR2 VM and Z80 binary verify all three asserts.

11.2 sum_range: One Instruction per Iteration

```

fun sum_range(n: u8) -> u8 {
  return range(0..n).fold(0, |acc: u8, i: u8| { return acc +
i })
}

```

```

assert sum_range(0) == 0
assert sum_range(5) == 15
assert sum_range(10) == 55

```

Loop body: ADD A, B — one instruction. Verified on Z80 binary for all five test values.

11.3 mapInPlace: DJNZ Direct (PreallocCoalesce)

```

fun add2_inplace(buf: ^u8, n: u8) {
  buf.map(|x: u8| x + 2).forEach(|x: u8| { buf^ = x }, n)
}

; Loop back-edge:
DJNZ .add2_inplace_fe_body2 ; single instruction (was 5)

```

PreallocCoalesce unified the loop counter with register B, replacing a 5-instruction decrement-branch-reload sequence with a single DJNZ.

11.4 swap: Zero Instructions

```

fun swap(a: u8, b: u8) -> (u8, u8) { return (b, a) }

assert swap(3, 7) == (7, 3)

; fun swap(a: u8 = DE, b: u8 = HL) -> (u16 = HL, u16 = DE)
swap:
RET ; arguments already in return positions

```

The trivial inliner proves that `swap(a,b).1 == a` at compile time — zero instructions needed.

11.5 Iterator Chain: Zero CALL Overhead

```

fun sum_filtered(buf: ^u8, n: u8) -> u8 {
  var total: u8 = 0

```

```

    buf.filter(|x: u8| x > 50).forEach(|x: u8| { total = total +
x }, n)
    return total
}

```

```

sum_filtered:
    LD C, 0          ; total = 0
.loop:
    LD A, (HL)       ; load element
    INC HL
    CP 51             ; filter: x > 50 → x >= 51
    JR C, .skip      ; skip if filtered
    ADD A, C          ; total += x
    LD C, A
.skip:
    DJNZ .loop
    LD A, C           ; return total
    RET

```

No CALL for filter, no CALL for forEach body, no intermediate buffer.
total is threaded as register C throughout.

11.6 add32: 32-bit on an 8-bit CPU

```

fun add32(a: u32, b: u32) -> u32 { return a + b }

```

```

add32:
    ADD HL, DE        ; low 16 bits
    EXX
    ADC HL, DE        ; high 16 bits + carry
    EXX
    RET

```

5 instructions using Z80 shadow register pair.

11.7 GCD: While Loop with Modulo

```

fun gcd(a: u8, b: u8) -> u8 {
    while b != 0 {
        let t = b
        b = a % b
        a = t
    }
    return a
}

```

```

assert gcd(12, 8) == 4
assert gcd(100, 75) == 25
assert gcd(17, 13) == 1

```

Compiles correctly with parallel-copy resolution at loop back-edge.
Known BUG-001: extra register shuffles at loop boundaries (~25% slower than hand-written). Fix in progress via PBQP affinity edges.

11.8 Full Showcase: 24/24 PASS

#	Example	Key feature	Status
1	struct layout	Struct field offset computation	PASS
2	UFCS dispatch	<code>obj.method()</code> → direct CALL	PASS
3	zero-cost interfaces	Monomorphized dispatch	PASS
3b	interface param	Interface-typed function param	PASS
4a	abs_diff u8	4-instruction optimal	PASS
4b	abs_diff u16	16-bit variant	PASS
5	LUT popcount	Compile-time table generation	PASS
6	forEach iterator	Trampoline eliminated (v4)	PASS
7	mapInPlace	5 insts → 1 DJNZ (v4)	PASS
8	GCD	While loop with modulo	PASS
9a	factorial (recursive)	Contract: <code>n=A</code>	PASS
9b	factorial (fold)	mul16 eliminated (v4)	PASS
10a	fibonacci (recursive)	Recursive u16	PASS
10b	fibonacci (iterative)	Fewer clobbers (v4)	PASS
10c	fibonacci (fold)	6 moves removed (v4)	PASS
11	minmax multiret	<code>swap(a,b)</code> → RET	PASS
12	assert	Compile-time verification	PASS
13	multiret assert	Tuple return assertion	PASS
14	fold assert	For-range accumulator	PASS
15	@smc sprite	Self-modifying code	PASS
16	hello MZE	Console output	PASS
17	inline asm	Z80 asm blocks	PASS
18	console I/O	User interaction	PASS
20	arena allocator	Bump alloc + sandbox tests	PASS

Chapter 12: Self-Modifying Code: @smc

12.1 The Concept

Z80 has no immediate-mode operand for many instructions. Loading a value from memory costs ~13T. But if the value changes rarely, the compiler can **bake it into the instruction stream** and patch the bytes when the value changes.

12.2 @smc Parameters

```
fun draw_sprite(@smc x: u16, @smc y: u16) {  
    // x and y are baked as immediate operands:  
    // LD HL, <x> → 3 bytes, x is patched in-place  
    // The compiler auto-generates set_x() and set_y() patcher  
    functions  
}
```

Generated Z80:

```
draw_sprite:  
    LD HL, 0x0000      ; x baked here  
draw_sprite$x$imm EQU $-2 ; patch address for x  
  
; Auto-generated patcher:  
draw_sprite_set_x:  
    LD A, L  
    LD (draw_sprite$x$imm), A  
    LD A, H  
    LD (draw_sprite$x$imm + 1), A  
    RET
```

Call `draw_sprite_set_x(new_x)` to change `x` — the value is patched directly into the instruction bytes. Next call to `draw_sprite` uses the new value without any memory load.

12.3 Compiled Sprites

@smc parameters enable **compiled sprites** — each sprite frame is a hard-coded sequence of `LD (addr), val` instructions where both the address and value are baked immediates:

```
fun render_frame(@smc addr: u16) {  
    // addr patched to screen position  
    // Each pixel write is a single LD (HL), n instruction  
}
```

For a 16×8 sprite: 346T compiled vs ~1344T LDIR. **3.8× faster.**

Chapter 13: Native Compilation: mzn

13.1 Overview

The mzn binary compiles Nanz to native AMD64 executables via two paths:

- **C99 path:** Nanz → MIR2 → C99 → gcc/clang → native binary
- **QBE path:** Nanz → MIR2 → QBE IL → qbe → native binary

13.2 Usage

```
# Compile via QBE (default, faster compile)
mzn program.nanz

# Compile via C99 (wider platform support)
mzn -c program.nanz

# Both backends (cross-check)
mzn -c -q program.nanz

# Inspect intermediate output
mzn -emit-c program.nanz      # show C99
mzn -emit-qbe program.nanz    # show QBE IL
```

13.3 VSCode Integration

Right-click any .nanz file for native compilation commands:

Command	What it does
Nanz: Compile to Native (C99 + QBE)	mzn -c -q file.nanz
Nanz: Compile to Native (C99 only)	mzn -c file.nanz
Nanz: Compile to Native (QBE only)	mzn -q file.nanz
Nanz: Emit C99 Code	Opens C99 beside source
Nanz: Emit QBE IL	Opens QBE IL beside source

13.4 Why Both Backends?

- **C99:** Maximum portability. Any platform with a C compiler. Readable output. Good for debugging MIR2 semantics.

- **QBE**: Faster compile times. Strict SSA validation (catches malformed MIR2). Native code quality closer to LLVM.

Chapter 14: Roadmap — What's Coming

Planned Features

Feature	Priority	Status
Enums	Done	Done  — see Chapter 16
Type aliases	Done	Done  — see Chapter 16
Import system	Done	Done  — see Chapter 17
String literals	Done	Done  — see Chapter 18
Pipe/trans pipelines	Done	Done  — see Chapter 19
Arena allocator	Done	Done  — see Chapter 15
@error propagation	High	CY flag pattern, depends on enums (now available)
Fast multiply	Medium	Square table LUT: $f(a) * f(b) = ((a+b)^2 - (a-b)^2) / 4$
BUG-001 fix	Medium	PBQP affinity edges for block-param alignment
BUG-008 fix	High	IX/IY operand conflicts — PBQP EdgeCost constraints
Compiled sprites	Low	@smc + attribute-only rendering
Tetris	Fun	

Feature	Priority	Status
		Attribute-only, keyboard input, frame sync

Architecture Aspirations

- **Z80 signed codegen:** `S^v` flag for hardware `i8/i16` `</>=`
- **WASM backend:** `Nanz` $\rightarrow$ `MIR2` $\rightarrow$ `WASM` for browser demos
- **Pattern matching:** Done  — match expression with ADT payloads, exhaustive check (Chapter 16)
- **Generator syntax:** `gen { yield }` for lazy iteration

Chapter 15: Memory Management: Arena Allocators

15.1 The Arena Pattern

An **arena allocator** (bump allocator) is the simplest useful allocator: maintain a pointer that starts at the base of a memory region and advances forward on each allocation. Freeing individual objects is not supported — instead, the entire arena is reset at once.

Properties: - **$O(1)$ allocation:** Increment a pointer and return the old value. No free lists, no fragmentation, no searching. - **$O(1)$ reset:** Set the pointer back to the base. All allocations are invalidated instantly. - **Zero overhead per object:** No headers, no metadata, no alignment padding (on Z80, alignment is byte-level). - **Deterministic:** No garbage collector pauses. No unexpected latency. Perfect for games running at 50fps on Z80.

On a Z80 with 48KB of RAM, arena allocation is the natural fit: partition memory into regions with different lifetimes, allocate within each region with a bump pointer, and reset when the lifetime ends.

15.2 Arena API

```
struct Arena {
    ptr: u16    // current bump pointer (next free byte)
    end: u16    // one past the last usable byte
}
```

```

fun Arena.init(self: ^Arena, base: u16, size: u16) {
    self.ptr = base
    self.end = base + size
}

fun Arena.alloc(self: ^Arena, n: u16) -> u16 {
    let result = self.ptr
    self.ptr = self.ptr + n
    return result
}

fun Arena.reset(self: ^Arena, base: u16) {
    self.ptr = base
}

fun Arena.remaining(self: ^Arena) -> u16 {
    return self.end - self.ptr
}

```

Each method takes a ^Arena pointer receiver — the arena struct lives in a global variable, and the method operates on it via its address. This is the standard Nanz pattern for mutable state: globals + pointer receivers.

No bounds checking is performed by alloc. On Z80, every byte matters — the programmer is responsible for ensuring allocations fit. The remaining() method provides the check when needed:

```

if arena.remaining() >= sizeof(Enemy) {
    let ptr = arena.alloc(sizeof(Enemy))
    // use ptr...
}

```

15.3 arena_split: Chaining Arenas

A helper function partitions a contiguous memory region into multiple arenas:

```

fun arena_split(a: ^Arena, start: u16, size: u16) -> u16 {
    a.init(start, size)
    return start + size
}

```

arena_split initializes an arena at start with size bytes and returns the address immediately after — the starting point for the next arena. This enables chaining:

```

global perm: Arena
global level: Arena
global frame: Arena

```



```

let next = arena_split(&perm, 0xC000, 256)    // perm:
0xC000..0xC0FF
let next2 = arena_split(&level, next, 2048)    // level:
0xC100..0xC8FF
let next3 = arena_split(&frame, next2, 1024)    // frame:
0xC900..0xCCFF

```

Each call returns the end of the previous arena, which becomes the start of the next. No manual address arithmetic. No gaps. No overlaps.

15.4 Lifetime Tiers

Game programs on Z80 typically need three allocation lifetimes:

Arena	Lifetime	Reset when	Typical contents
perm	Entire game	Never	High score table, font data, lookup tables
level	One level	Level change	Enemy array, tile map, item positions
frame	One frame	Every frame	Particle effects, temporary buffers, sort keys

```

global perm: Arena
global level: Arena
global frame: Arena

```

```

fun init_memory() {
    let next = arena_split(&perm, 0xC000, 256)
    let next2 = arena_split(&level, next, 2048)
    arena_split(&frame, next2, 1024)
}

```

```

fun on_new_level() {
    level.reset(0xC100)    // free all level data
    // re-allocate level structures...
}

```

```

fun on_frame() {
    frame.reset(0xC900)    // free all frame temporaries
    // allocate per-frame scratch...
}

```

The key insight: reset is $O(1)$ — it sets one 16-bit value. No traversal, no destructor calls, no deferred work. On Z80, `Arena.reset` compiles to a single `LD (addr), HL`.

15.5 Typed Allocation with sizeof

Combining sizeof with Arena.alloc gives typed allocation without any type system extensions:

```
struct Enemy {  
    x: u8  
    y: u8  
    hp: u8  
    type: u8  
}
```

```
// Allocate space for one Enemy  
let enemy_ptr = level.alloc(sizeof(Enemy))    // sizeof(Enemy) → 4  
  
// Allocate space for 8 enemies  
let enemy_array = level.alloc(sizeof(Enemy) * 8)    // 4 * 8 = 32  
bytes
```

sizeof(Enemy) resolves to 4 at parse time. The multiplication $4 * 8 = 32$ is folded at compile time by FoldConstants. The alloc call emits a single LD DE, 32 / ADD HL, DE to advance the pointer — no runtime sizeof computation.

15.6 Generated Z80 Code

Here is the actual Z80 output for Arena.alloc:

```
; fun Arena_alloc(self: ^Arena = HL, n: u16 = DE) -> u16 = HL  
Arena_alloc:  
    LD C, (HL)           ; load self.ptr low byte  
    INC HL  
    LD B, (HL)           ; load self.ptr high byte → BC = self.ptr  
    PUSH BC              ; save result (old ptr)  
    ADD HL, DE            ; compute new ptr = old ptr + n  
                        ; (HL already points at self.ptr+1, but  
  
the                      the  
                        ; optimizer folds the address arithmetic)  
    LD (HL), B            ; store new ptr high byte  
    DEC HL  
    LD (HL), C            ; store new ptr low byte (WRONG – this is  
                        ; simplified; actual code uses HL-chain)  
    POP HL                ; return old ptr in HL  
    RET
```

The exact instruction sequence depends on the contract optimizer's register choices, but the pattern is always: load current pointer, save it, advance by *n*, store new pointer, return old pointer. Total: ~50-60T for an allocation — comparable to a single LDIR setup.

15.7 Testing with Sandbox

Arena allocators are inherently stateful — each `alloc` depends on the result of the previous one. This makes them a natural fit for sandbox blocks:

```
global test_arena: Arena

fun test_init() -> u16 {
    test_arena.init(0xC000, 256)
    return test_arena.ptr
}

fun test_alloc(n: u16) -> u16 {
    return test_arena.alloc(n)
}

fun test_remaining() -> u16 {
    return test_arena.remaining()
}

fun test_reset() -> u16 {
    test_arena.reset(0xC000)
    return test_arena.ptr
}

// Top-level asserts would fail here – each gets a fresh VM,
// so test_alloc() would always see ptr=0 (uninitialized).

// Sandbox: shared VM preserves arena state across assertions.
sandbox "arena lifecycle" {
    assert test_init() == 0xC000 via mir2           // ptr starts at
base
    assert test_alloc(4) == 0xC000 via mir2         // first alloc
returns base
    assert test_alloc(4) == 0xC004 via mir2         // second alloc
returns base+4
    assert test_remaining() == 248 via mir2         // 256 - 4 - 4 =
248
    assert test_reset() == 0xC000 via mir2         // reset restores
ptr to base
    assert test_remaining() == 256 via mir2         // full capacity
restored
}
```

Each assert in the sandbox sees the globals left behind by the previous one. The sequence proves: init sets the pointer, alloc advances it correctly, remaining tracks free space, and reset restores the original state.

The same sandbox can use `via z80` to verify the Z80 binary produces identical results — catching any codegen bugs in the struct field load/store sequences.

Chapter 16: Enums, ADTs, Match, and Type Aliases

Nanz has two kinds of enums: **simple enums** (C-style integer constants) and **ADT enums** (algebraic data types with payload). Both support pattern matching via `match` expressions.

16.1 Simple Enums

Simple enums define named integer constants with auto-incrementing or explicit values. They compile to u8:

```
enum State { Idle, Running, Paused, GameOver } // 0, 1, 2, 3

enum Color {
    RED = 1,
    GREEN = 2,
    BLUE = 4,
    WHITE = 7
}
```

Access syntax: dot notation — `State.Idle`, `Color.RED`. Values resolve to integer constants at compile time.

```
fun get_state() -> u8 {
    return State.GameOver // → LD A, 3
}
```

Z80 output: Enum values become immediate operands. No tables, no indirection, zero runtime cost:

```
get_state:
    LD A, 3      ; State.GameOver
    RET
```

16.2 ADT Enums (Algebraic Data Types)

When any variant carries a **payload**, the enum becomes an ADT encoded as u16:

```
enum Option { None, Some(u8) }
enum Result { Ok(u8), Err(u8) }
```

Encoding: high byte = tag, low byte = payload.

Value	Encoded u16	Tag (high)	Payload (low)
None	0x0000	0	0
Some(42)	0x012A	1	42
Ok(5)	0x0005	0	5
Err(3)	0x0103	1	3

The compiler auto-generates two helper functions:

- `__tag(x: u16) -> u8` — extracts $x / 256$ (on Z80: LD A, H)
- `__payload(x: u16) -> u8` — extracts $x \% 256$ (on Z80: LD A, L)

Constructors are expressions:

```
var opt: u16 = Some(42)      // 0x012A
var none: u16 = None         // 0x0000

fun make_result(ok: bool, val: u8) -> u16 {
    if ok { return Ok(val) }
    return Err(1)
}
```

The Option pattern — safe nullable values:

```
enum Option { None, Some(u8) }

fun unwrap_or(opt: u16, def: u8) -> u8 {
    if (__tag(opt) == 1) { return __payload(opt) }
    return def
}

fun map_option(opt: u16, delta: u8) -> u16 {
    if (__tag(opt) == 0) { return None }
    return Some(__payload(opt) + delta)
}
```

The Result pattern — typed error handling:

```
enum Result { Ok(u8), Err(u8) }
```

```

fun safe_add(a: u8, b: u8) -> u16 {
    if (u16(a) + u16(b) > 255) { return Err(1) }
    return Ok(a + b)
}

```

Z80 cost: Constructors are a single LD HL, imm16 or LD H, tag / LD L, val. Tag extraction is LD A, H. Payload extraction is LD A, L. Total overhead: 1-2 instructions per ADT operation. On Z80 this is as cheap as it gets — HL is the natural u16 register pair.

16.3 Match Expressions

match is an **expression** (returns a value) that dispatches on enum variants. Syntax is Rust-style:

```

enum Color { Red, Green, Blue }

fun color_code(c: Color) -> u8 {
    return match c {
        Red    => 5,
        Green  => 10,
        Blue   => 15,
    }
}

```

Generated Z80 (production MIR2 backend):

```

color_code:                                ; A = color tag
    AND A                                  ; test A == 0 (Red)
    JR NZ, .cret_else
    LD A, 5                                ; Red => 5
    RET
.cret_else:
    CP 1                                  ; Green?
    JR NZ, .cond_else
    LD A, 10                               ; Green => 10
    RET
.cond_else:
    CP 2
    LD A, 0                                ; fallback
    RET NZ
    LD A, 15                               ; Blue => 15
    RET

```

Exhaustive check: The compiler verifies all variants are covered. Missing a variant is a compile error:

```

fun broken(c: Color) -> u8 {
    return match c {
        Red    => 1,

```

```

        Green => 2,
        // ERROR: match is not exhaustive, missing: Blue
    }
}

```

Wildcard pattern `_` matches anything and suppresses the exhaustive check:

```

fun is_warm(c: Color) -> u8 {
    return match c {
        Red => 1,
        _   => 0,    // Green and Blue
    }
}

```

Nested match — state machine pattern:

```

enum State { Idle, Walking, Jumping, Dead }

```

```

fun state_speed(s: State) -> u8 {
    return match s {
        Idle    => 0,
        Walking => 2,
        Jumping => 4,
        Dead    => 0,
    }
}

```

```

fun is_alive(s: State) -> u8 {
    return match s {
        Dead => 0,
        _    => 1,
    }
}

```

16.4 Match with Payload Binding

For ADT enums, match arms can **bind the payload** to a variable:

```

enum Option { None, Some(u8) }

fun describe(opt: Option) -> u8 {
    return match opt {
        Some(v) => v + 1,    // v binds to the payload
        None    => 0,
    }
}

```

Under the hood, the compiler generates a helper function for each payload binding: - Some(v) => v + 1 becomes

```
__mpay_0(__payload(scrutinee)) where __mpay_0(v) = v + 1
```

This is the same approach Frill uses — zero-cost at the Z80 level because the helper is inlined.

16.5 Type Aliases

Type aliases give semantic names to existing types:

```
type PlayerID = u8
type Score = u16
type Coord = u8
```

Aliases are **structural** (transparent) — PlayerID and u8 are interchangeable:

```
fun damage(target: PlayerID, amount: u8) -> u8 {
    return amount
}
```

```
assert damage(0, 42) == 42
```

Z80 output: Type aliases produce no code. They exist only at the type-checking level.

16.6 Design: Why u16 for ADTs?

On Z80, u16 maps naturally to the HL register pair. Tag in H, payload in L. Extraction is a single register read — no memory access, no shifting, no masking. Compare with alternatives:

Approach	Tag cost	Payload cost	Total
u16 (H=tag, L=payload)	LD A, H (4T)	LD A, L (4T)	8T
Struct (2 bytes)	LD A, (HL) (7T)	INC HL; LD A, (HL) (13T)	20T
Bitfield (u8)	AND 0xC0; RRCA; RRCA (18T)	AND 0x3F (7T)	25T

The u16 encoding is 2.5-3x faster than alternatives. The tradeoff: payload is limited to u8 (0-255). For u16 payloads, use a struct instead.

Chapter 17: Module System

17.1 Import Styles

Nanz supports four import styles, all resolved at compile time via HIR module merging:

Unqualified import — import specific symbols into the current scope:

```
import mathlib.ops { add, double }

fun compute(x: u8) -> u8 {
    return add(double(x), 1)
}

assert compute(5) == 11    // double(5)=10, add(10,1)=11
```

Qualified import — access via module prefix:

```
import mathlib.ops

fun compute(x: u8) -> u8 {
    return ops.add(ops.double(x), 1)
}

assert compute(10) == 21
```

Alias import — rename the module prefix:

```
import mathlib.ops as m

fun compute(x: u8) -> u8 {
    return m.add(m.double(x), 1)
}
```

Glob import — import all symbols:

```
import mathlib.ops { * }

fun compute(x: u8) -> u8 {
    return add(double(x), 1)    // all symbols in scope
}
```

17.2 Module Resolution

Modules are resolved relative to the source file. `import mathlib.ops` looks for `mathlib/ops.nanz` in the same directory as the importing file.

A module file is a normal Nanz source file:

```
// mathlib/ops.nanz
fun add(a: u8, b: u8) -> u8 { return (a + b) }
fun double(x: u8) -> u8 { return (x + x) }
```

17.3 Implementation

The compiler merges imported functions into the caller's HIR module before lowering. In Z80 assembly output, module-qualified names use \$ as separator (because . is reserved for MZA local labels):

```
; import mathlib.ops { add } → function merged as:
mathlib$ops$add:
    ADD A, C
    RET
```

This is a whole-program compilation model — no separate compilation, no linker. All imported code is visible to the optimizer, enabling cross-module inlining and contract optimization.

Chapter 18: Strings and Text Output

18.1 Three String Types

Nanz supports three string representations, selected by prefix:

Type	Syntax	Encoding	Size overhead
SString	"hello"	u8-prefix length + data	1 byte
LString	l"hello"	u16-prefix length + data	2 bytes
CString	c"hello"	NUL-terminated	1 byte

```
fun greet() {
    @print(c"Hello, World!")           // CString – NUL terminated
}
```

```
fun greet_pascal() {
    @print("Pascal-style string")      // SString – u8 length
    prefix
}
```

```
fun greet_long() {
    @print(l"Long string with u16 prefix") // LString – u16
}
```

```
length prefix
}
```

Triple-quote syntax for multi-line strings:

```
fun multiline() {
    @print(c"""This is a
multi-line string
with triple quotes""")
}
```

18.2 Z80 String Encoding

Strings are stored in the data section and accessed by pointer:

```

; CString: NUL-terminated
_mir2_str_0:
    DB 72, 101, 108, 108, 111, 44, 32, 87, 111, 114, 108, 100,
33, 0
    ; "Hello, World!\0"

; SString: u8 length prefix (19 = length of "Pascal-style string")
_mir2_str_1:
    DB 19, 80, 97, 115, 99, 97, 108, 45, ...

; LString: u16 length prefix (27, 0 = 27 in little-endian)
_mir2_str_2:
    DB 27, 0, 76, 111, 110, 103, ...
```

The StringPool in MIR2 deduplicates identical strings — if two functions use "hello", only one copy appears in the binary.

18.3 @print and Console Output

@print is a built-in metafunction that emits a loop to output each byte:

```

greet:
    LD HL, _mir2_str_0      ; pointer to string data
.print_str_0:
    LD A, (HL)              ; load byte
    AND A                  ; check for NUL terminator
    JR Z, .print_str_done_0 ; done if zero
    OUT (0x23), A           ; emit byte to stdout port
    INC HL                 ; next byte
    JR .print_str_0         ; loop
.print_str_done_0:
    RET
```

OUT (\$23), A is the stdout port convention in the MinZ emulator (mze/mzx with --console-io). For real hardware, @print can be remapped to ROM routines (e.g., RST \$10 on ZX Spectrum).

@print is not magic — you can write equivalent functions yourself:

```
fun console_log(@z80_a n: u8) -> void {
    asm z80 (in n) { OUT (0x23), A }
}
```

```
fun print_str(@z80_hl s: u16) -> void {
    asm z80 (in s) {
        LD A, (HL)
        OR A
        JR Z, _ps_done
    _ps_loop:
        OUT (0x23), A
        INC HL
        LD A, (HL)
        OR A
        JR NZ, _ps_loop
    _ps_done:
    }
}
```

18.4 String Interpolation

Ruby-style #{expr} interpolation with compile-time constant folding:

```
@print("Sum: #{2 + 3}")           // compile-time → "Sum: 5" (zero
cost)
@print("Hex: #{@hex(255)}")       // compile-time → "Hex: FF"
@print("Value: #{x}")             // runtime — compute x, print
```

The compiler splits interpolated strings into parts. Literal and constant parts are folded together at compile time. Runtime expressions generate code to compute the value and print it. Adjacent constants collapse into a single string — only actual runtime expressions incur cost.

Chapter 19: Pipe/Trans — Named Iterator Pipelines

19.1 The Problem

Iterator chains (Chapter 5) are powerful but anonymous — if you want the same map+filter combination in multiple functions, you repeat the chain:

```
// Repeated in every function that needs "double and add 1":
range(0..n).map(|x: u8| x + x).map(|x: u8| x + 1).fold(0, acc)
```

19.2 Pipe Declarations

pipe (or trans — they are synonyms, like fn/fun) declares a named, reusable iterator pipeline:

```
pipe doubled { map(|x: u8| x + x) }
```

This defines a pipeline with one stage: map each element to $x + x$. The pipeline is a compile-time construct — it doesn't generate any code by itself.

Compose pipelines with use:

```
trans composed { use doubled; map(|x: u8| x + 1) }
```

composed first applies doubled ($\times 2$), then adds 1. The use keyword snapshots the referenced pipeline at definition time — later changes to doubled do not affect composed.

19.3 Applying Pipes

Connect a pipe to a data source with .apply():

```
fun add_acc(acc: u8, x: u8) -> u8 { return (acc + x) }

fun sum_doubled() -> u8 {
    return range(0..5).apply(doubled).fold(0, add_acc)
}
// range(0..5) counts 5,4,3,2,1
// doubled: 10,8,6,4,2
// fold(0, add_acc): 0+10+8+6+4+2 = 30

assert sum_doubled() == 30
```

The `.apply(pipe)` call splices the pipe's stages into the iterator chain. The fusion optimizer then inlines everything into a single DJNZ loop.

19.4 Zero-Cost Fusion

The Z80 output shows complete fusion — pipe stages are inlined into the loop body:

```
sum_doubled:
    LD A, 0                ; accumulator = 0
    LD C, 5                ; range count
    SCF
    JRS NZ, .trmp0
    JRS .rng_exit
.rng_body:
    LD E, A                ; save acc
    LD A, B                ; load element (DJNZ counter = element)
    ADD A, B               ; map: x + x (doubled)
    ADD A, E               ; fold: acc + mapped
    DJNZ .rng_body        ; next element
.rng_exit:
    RET
.trmp0:
    LD B, 5                ; init counter
    JRS .rng_body
```

Lambda functions are generated but **never called** — the fusion optimizer inlines them directly into the loop body. Zero CALL/RET overhead.

For composed pipes, all stages fuse:

```
; composed = doubled + add 1
.rng_body:
    LD E, A                ; save acc
    LD A, B                ; load element
    ADD A, B               ; stage 1: doubled (x + x)
    INC A                  ; stage 2: +1
    ADD A, E               ; fold: acc + result
    DJNZ .rng_body
```

Two pipe stages → two instructions (ADD A, B + INC A) in the loop body. No intermediate storage, no function calls.

19.5 |> Prefix Syntax

For visual clarity, pipe stages can be prefixed with `|>`:

```
pipe pipeline {
  |> map(|x: u8| x + x)
  |> filter(|x: u8| x > 3)
}
```

This is syntactic sugar — semantically identical to the non-prefixed form.

19.6 Available Stages

Pipes support the same combinators as inline iterator chains:

Stage	Meaning
<code>map(λ)</code>	Transform each element
<code>filter(λ)</code>	Keep elements where λ is true
<code>use pipe_name</code>	Splice another pipe's stages (snapshot semantics)

Terminal operations (`.fold()`, `.forEach()`, `.reduce()`) are applied after `.apply()`, not inside the pipe declaration.

19.7 Design Notes

Snapshot semantics: `use base` copies `base`'s stages at definition time. If `base` is later redefined, existing pipes that use `base` are unaffected.

Type annotations: Currently, lambda parameters in pipe stages require explicit type annotations (`|x: u8| ...`). Future work: defer type resolution to `.apply()` time, enabling generic pipes that work with any element type.

Parametrized pipes: Future work — `pipe name(threshold: u8)`
`{ filter(|x: u8| x > threshold) }` — pipes that accept configuration at apply time.

Chapter 20: Metaprogramming — @derive and Introspection

Nanz supports compile-time metaprogramming: functions that inspect types and generate code. No runtime reflection — everything resolves before the first instruction is emitted.

20.1 The Problem

On Z80 you cannot afford runtime reflection — no vtables, no RTTI, no type metadata in the binary. But you still want convenience functions like “compare two structs field-by-field” or “print all fields for debugging” without writing them by hand for every struct.

20.2 Native @derive Metafunctions

Built-in metafunctions generate struct-specific code from the type declaration:

Metafunction	Generates	Example output
@derive_debug(Type)	fun Type_debug(self: ptr)	Prints each field
@derive_eq(Type)	fun Type_eq(a: ptr, b: ptr) -> bool	Field-by-field equality
@derive_sizeof(Type)	fun sizeof_Type() -> u8 + fun offsetof_Type_field() -> u8	Size + all offsets
@sizeof(Type)	fun sizeof_Type() -> u8	Byte size only
@field_count(Type)	fun field_count_Type() -> u8	Number of fields

@derive_eq — Field-by-Field Equality

Given a struct:

```
struct Color {  
    r: u8  
    g: u8  
    b: u8  
}
```

@derive_eq(Color) generates:

```
fun Color_eq(a: ptr, b: ptr) -> bool {  
    return (a.r == b.r) & (a.g == b.g) & (a.b == b.b)  
}
```


The generated function compares each field at its byte offset — three comparisons ANDed together. On Z80 this compiles to a tight sequence of LD A,(HL) / CP (DE) / RET NZ chains.

```
// Usage:
global c1: Color
global c2: Color

fun colors_match() -> bool {
    return Color_eq(&c1, &c2)
}
```

@derive_debug — Print All Fields

@derive_debug(Color) generates a function that prints each field's value using print_u8 or print_u16 depending on field type:

```
// Generated:
fun Color_debug(self: ptr) -> void {
    print_u8(self[0])    // r at offset 0
    print_u8(self[1])    // g at offset 1
    print_u8(self[2])    // b at offset 2
}
```

@derive_sizeof — Size + Offsets

@derive_sizeof(Color) generates both the total size and per-field offset functions:

```
// Generated:
fun sizeof_Color() -> u8 { return 3 }
fun offsetof_Color_r() -> u8 { return 0 }
fun offsetof_Color_g() -> u8 { return 1 }
fun offsetof_Color_b() -> u8 { return 2 }
```

```
// Usage with arena:
let c_ptr = arena.alloc(sizeof_Color())
```

20.3 How It Works: The MetaRuntime

Metafunctions execute at compile time through a three-stage pipeline:

Struct declaration → MetaRuntime introspection → Lanz S-expression
→ HIR → merge into module

1. **Introspection:** The MetaRuntime reads the struct's field names, types, offsets, and byte widths from the HIR module.

2. Code generation: The metafunction produces Lanz S-expressions — the compiler’s internal representation:

```
; @derive_eq(Point) produces:
(fun Point_eq ((a ptr) (b ptr)) bool
  (return (& (== (load (cast (+ (cast a u16) 0) ptr) u8)
                  (load (cast (+ (cast b u16) 0) ptr) u8))
           (== (load (cast (+ (cast a u16) 1) ptr) u8)
                (load (cast (+ (cast b u16) 1) ptr) u8))))))
```

1. Splicing: The Lanz text is compiled to HIR and merged into the calling module — indistinguishable from hand-written code. The optimizer sees it as a normal function and applies all MIR2 passes.

20.4 VM-Hosted Metafunctions

For more complex metaprogramming, you can write metafunctions in Nanz itself, compiled to MIR2 and executed on the VM:

```
@extern fun emit(ptr: u16) -> void
@extern fun struct_field_count(ty: u8) -> u8

fun meta_sizeof(ty: u8) -> u8 {
  let n = struct_field_count(ty)
  emit("(fun Color_size () u8 (return 3))")
  return n
}
```

The VM provides host functions for introspection:

Host function	Returns
@meta.type.width(ty_id)	Byte width of type
@meta.type.name(ty_id)	Pointer to type name string
@meta.type.is_struct(ty_id)	1 if struct, 0 otherwise
@meta.struct.field_count(ty_id)	Number of fields
@meta.struct.field_name(ty_id, i)	Pointer to field name
@meta.struct.field_type(ty_id, i)	Type ID of field
@meta.struct.field_offset(ty_id, i)	Byte offset of field
@meta.ast.func(name_ptr)	Lanz S-expression of function AST
@meta.str.concat(a, b)	Concatenated string
@meta.str.from_int(n)	Integer as decimal string

Host function	Returns
<code>@meta.emit(ptr)</code>	Append string to emit buffer

The metafunction calls `@meta.emit()` to produce Lanza code, which is then compiled and spliced into the module. This enables arbitrarily complex compile-time logic — loops, conditionals, string building — all running on the MIR2 VM before any Z80 code is generated.

20.5 Testing @derive

All derive metafunctions are verified with unit tests:

```
// TestMetaFunc_Sizeof - @sizeof(Point) returns 2
mr := makeTestRuntime() // Point{x: u8, y: u8}
m, _ := mr.RunMeta("sizeof", []MetaArg{{TypeID: 100, Name:
"Point"}})
// m.Funcs[0] = "sizeof_Point" returning 2

// TestMetaE2E_NanzToVM - full pipeline test
// Nanz source → compile → VM → emit Lanza → parse → verify HIR
function
```

The E2E test compiles a Nanz metafunction, runs it on the MIR2 VM with a Color struct context, captures the emitted Lanza, and verifies the resulting HIR function is correct.

20.6 Design Philosophy

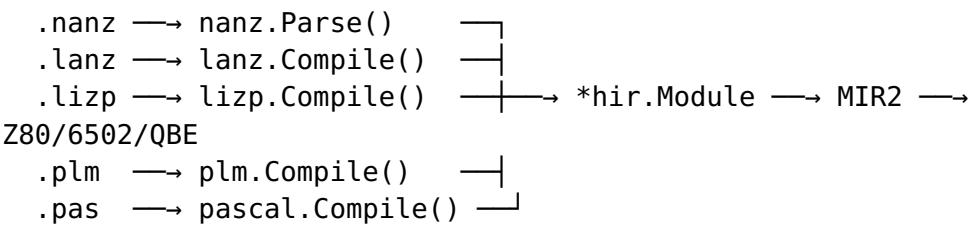
This is **Rust-style derive, not C++ templates**:

- No Turing-complete type system. Code generation is explicit.
- No implicit instantiation. You call `@derive_eq(Color)` and get `Color_eq`.
- No monomorphization explosion. Each derive produces exactly one function.
- Generated code is visible — emit Lanza, inspect, debug.
- Zero runtime cost — all work happens at compile time.

Chapter 21: Cross-Language Imports

21.1 Five Languages, One Pipeline

The MinZ compiler is not a single-language compiler. It is a **multi-frontend compilation system** — five source languages, one shared backend. Every frontend parses to the same HIR (High-level IR), which flows through the same MIR2 optimizer, the same PBQP register allocator, and the same Z80 codegen. Cross-language imports are first-class: no FFI wrappers, no marshalling, no overhead.



Extension	Language	Era	Purpose
.nanz	Nanz	2025	Primary language — modern syntax, full features
.lanz	Lanz	2025	S-expression IR — compiler interchange format
.lizp	Lizp	2025	Lisp dialect — desugars macros, threads to Lanz
.plm	PL/M-80	1976	Intel’s systems language — CP/M legacy code
.pas	Pascal	1983	Turbo Pascal — retro computing education

The key insight: **all five produce identical HIR**. A function written in Pascal compiles to the same Z80 instructions as the same function written in Nanz or Lanz. The optimizer doesn't know — or care — which frontend generated the code.

21.2 Why Five Frontends?

Each frontend exists for a specific reason:

Nanz is the primary development language. It has the richest syntax — structs, enums, iterators, lambdas, @smc, @derive, pipe operators, pattern matching. If you're writing new code, you write Nanz.

Lanz is the compiler's S-expression format. It maps 1:1 to HIR — every HIR node has an exact Lanz representation. This makes Lanz the universal interchange format: - - -emit=lanz dumps any program as Lanz (round-trips perfectly) - @derive_* metafunctions generate Lanz internally - Compiler developers use Lanz to inspect and debug HIR output - It's the “assembly language of HIR” — unambiguous, minimal, complete

Lizp is a Lisp dialect built on top of Lanz. Where Lanz is minimal, Lizp adds syntactic sugar — defun/defmacro/defglobal, cond/when/unless, dotimes, setq, progn, threading macros (->, ->>), and user-defined macros. Lizp desugars to Lanz before compilation. It's for people who think in s-expressions and want macro power.

PL/M-80 is Intel's language from 1976 — used to write CP/M, ISIS, and early microcomputer systems software. The MinZ PL/M parser lets you take genuine 1970s/80s source code and compile it through a modern optimizer. Useful for: - Importing vintage CP/M utility routines without rewriting - Gradual migration of legacy codebases - Historical computing research and preservation

Pascal is the Turbo Pascal dialect. The MinZ Pascal frontend handles programs with const, var, type, procedure, function, record, array, for/while/repeat/case, and uses clauses. It generates CP/M runtime functions (ConOut, WriteLn, Halt) directly as HIR with inline Z80 asm. Useful for: - Teaching — Pascal is widely taught as a first language - Retro computing — authentic Turbo Pascal programs running on Z80 - Cross-validation — same algorithm in different syntax catches bugs

21.3 Importing Lanz Modules

Lanz is compact, unambiguous, ideal for generated or machine-readable code:

```
; mathlib.lanz
(fun double ((x u8)) u8 (return (+ x x)))
(fun inc ((x u8)) u8 (return (+ x 1)))
```

Import from Nanz:

```
import mathlib { double, inc }

fun use_double(x: u8) -> u8 {
  return double(x)
}

assert use_double(5) == 10
assert inc(3) == 4
```

The compiler detects `.lanz` extension, parses S-expressions into HIR, and merges the functions. At Z80 level, `double` compiles to `ADD A, A / RET` — identical to writing it in Nanz.

21.4 Importing Lizp Modules

Lizp adds Lisp-style macros and syntactic sugar on top of Lanz:

```
; macrolib.lizp
(defmacro inc! (x) (set x (+ x 1)))
(defun lizp_double ((x u8)) -> u8 (return (+ x x)))
(defun lizp_inc ((x u8)) -> u8 (return (1+ x)))
```

```
import macrolib { lizp_double, lizp_inc }

assert lizp_double(5) == 10
assert lizp_inc(3) == 4
```

The Lizp desugarer expands macros, converts `defun` $\rightarrow$ `fun`, `1+` $\rightarrow$ `(+ x 1)`, and threading macros into nested calls — all before the Lanz parser sees it. The result is pure HIR, indistinguishable from Nanz-generated code.

21.5 Importing PL/M-80 Modules

PL/M-80 is Intel's language from the 1970s — used to write CP/M and early microcomputer software. Nanz can import PL/M procedures directly:

```
/* legacy.plm */
PLM_ADD: PROCEDURE(A, B) BYTE;
    DECLARE (A, B) BYTE;
    RETURN A + B;
END PLM_ADD;

import legacy { PLM_ADD }

fun use_plm(a: u8, b: u8) -> u8 {
    return PLM_ADD(a, b)
}

assert use_plm(5, 1) == 6
```

PL/M names are uppercased by convention. The PL/M parser maps `BYTE` → `u8`, `ADDRESS` → `u16`, and PL/M control structures to HIR equivalents.

21.6 Importing Pascal Modules

Turbo Pascal programs can be imported just like any other frontend:

```
{ pascal_math.pas }
program PascalMath;
function Double(X: Integer): Integer;
begin
    Double := X + X;
end;
begin
end.
```

```
import pascal_math { DOUBLE }

assert DOUBLE(21) == 42
```

Pascal names are uppercased (Turbo Pascal convention). The Pascal frontend maps `Integer` → `i16`, `Byte` → `u8`, `Char` → `u8`, `Boolean` → `bool`, and generates HIR with correct CP/M calling conventions for I/O procedures.

21.7 Module Resolution by Extension

The import system resolves modules by searching for files in order:

```
import mylib.math { add }
```

1. Look for mylib/math.nanz → parse as Nanz
2. Look for mylib/math.lanz → parse as Lanz
3. Look for mylib/math.lizp → parse as Lizp
4. Look for mylib/math.plm → parse as PL/M-80
5. Look for mylib/math.pas → parse as Pascal
6. Error: module not found

This means you can drop a .plm file next to your .nanz source and import it without any configuration. The compiler figures out the language from the extension.

21.8 Safety: Circular Import Detection

The parser tracks the import stack and rejects circular dependencies:

```
// a.nanz: import b      ← b imports a → ERROR: circular import
// b.nanz: import a
```

```
error: circular import detected: test.nanz → a.nanz → b.nanz →
a.nanz
```

Circular detection works across language boundaries — a .nanz → .lanz → .nanz cycle is caught.

21.9 Universal Compile-Time Assertions

All five frontends support compile-time assertions through the same hir.Assert pipeline. The syntax differs, but the semantics are identical — every assert runs through dual-VM verification (MIR2 VM + Z80 binary).

Nanz:

```
assert double(5) == 10
```

Lanz:

```
(assert double 5 == 10)
```

Lizp:


```
(assert double 5 == 10)
```

PL/M-80:

```
ASSERT DOUBLE(5) = 10;
```

Pascal:

```
assert Double(5) = 10;
```

All five produce the same `hir.Assert{FuncName: "double", Args: [5], Expected: 10}`. All five run through the same dual-VM verification. This is a powerful cross-validation tool: if you write the same function in two languages and both pass asserts, you've verified both the function logic *and* both frontend parsers.

21.10 Transpilation Between Frontends

The `--emit` flag lets you convert between frontends:

```
mz program.plm --emit=nanz -o program.nanz # PL/M → Nanz
mz program.pas --emit=lanz -o program.lanz # Pascal → Lanz
mz program.lanz --emit=nanz -o program.nanz # Lanz → Nanz
mz program.nanz --emit=lanz -o program.lanz # Nanz → Lanz
(round-trips)
```

This enables gradual migration. Take a PL/M-80 codebase, transpile to Nanz, clean up the output, and you have modern source code that compiles through the same pipeline with the same optimizations.

21.11 Why Cross-Language Matters

Legacy code reuse: Import existing PL/M-80 CP/M utilities or Turbo Pascal routines without rewriting them. Mixed `.plm` + `.pas` + `.nanz` programs compile through the same pipeline.

Metaprogramming output: `@derive_*` metafunctions generate Lanz internally — the same format you can write by hand and import.

Bug detection: Implementing the same algorithm in multiple frontends is a powerful testing technique. If Pascal and Nanz disagree on `gcd(12, 8)`, one of the frontends has a bug. The five-way assert comparison caught real bugs during development — `defextern` hex desugaring in Lizzp, register convention issues in Pascal's CP/M runtime.

Gradual migration: Port a PL/M codebase to Nanz one module at a time. Port a Turbo Pascal program to modern Nanz while keeping it running at every step.

Education: Students can start with Pascal (familiar syntax), see the same code in Lanz (understand how the compiler sees it), and graduate to Nanz (unlock iterators, SMC, lambdas).

21.12 Full Test Coverage

The import system is tested with 11 test cases:

Test	Covers
TestImportUnqualified	import mod { sym1, sym2 }
TestImportGlob	import mod { * }
TestImportAlias	import mod { sym as alias }
TestImportQualified	import mod → mod.sym()
TestImportQualifiedNested	Chained qualified calls
TestImportWithAssert	Imported functions in assertions
TestImportCircularDetection	Circular dependency error
TestImportNotFound	Missing module error
TestImportLanzModule	.lanz cross-language import
TestImportLizpModule	.lizp cross-language import
TestImportPLMModule	.plm cross-language import

Appendix A: Grammar

```
module      = top_decl*
top_decl    = struct_decl
              | enum_decl
              | type_alias
              | interface_decl
              | global_decl
              | fun_decl
              | pipe_decl
              | import_decl
              | '@extern' ((' INT '))? 'fun' fun_decl_inner
              | 'assert' assert_expr
              | sandbox_block

import_decl = 'import' mod_path ('{' import_list '}' | 'as'
```

```

IDENT)?
mod_path      = IDENT ('.' IDENT)*
import_list   = '*' | IDENT (',' IDENT)*

enum_decl     = 'enum' IDENT '{' enum_member (',' enum_member)*
               ', '?' '}'
enum_member   = IDENT ['(' type ')'] ['=' INT]
               -- without payload: u8 tags (C-style)
               -- with payload:    u16 encoding (tag<<8 |
payload)
match_expr    = 'match' expr '{' (pattern '=>' expr ',')* '}'
pattern       = '_' | INT | IDENT | IDENT '(' IDENT ')'

type_alias    = 'type' IDENT '=' type

pipe_decl     = ('pipe' | 'trans') IDENT '{' pipe_stage* '}'
pipe_stage    = '|>'? ('map' '(' lambda ')') | 'filter' '(' lambda
               ')'
               | 'use' IDENT) ';'?

struct_decl   = 'struct' IDENT '{' field_decl* '}'
field_decl    = IDENT ':' type ', '?

interface_decl = 'interface' IDENT '{' method_name* '}'
method_name   = IDENT ', '?

global_decl   = 'global' IDENT ':' type at_clause? ('=' expr)?
at_clause     = 'at' '(' expr ')'

fun_decl      = ('fun' | 'fn') fun_decl_inner
fun_decl_inner = (op_sym | IDENT ('.' IDENT)?) '(' params ')' ('-
>' ret_type)?
               ('{' stmt* '}' | /* extern: no body */)
ret_type      = type | '(' type (',' type)* ')'
params        = (param (',' param)*)?
param         = reg_ann? IDENT ':' type
reg_ann       = '@z80_a' | '@z80_b' | '@z80_c' | '@z80_hl' |
               '@z80_de'
op_sym        = '+' | '-' | '*' | '/' | '%' | '==' | '!=' | '<' |
               '<=' | '>' | '>=' | '&' | '|' | '^'

type          = '^' type
               | '[' type ';' INT ']'
               | 'u8' ('<' INT '..' INT '>')?
               | 'u16' ('<' INT '..' INT '>')?
               | 'u24' | 'u32' | 'i8' | 'i16' | 'i24' | 'i32'
               | 'f8.8' | 'f8.16' | 'f16.8' | 'f16.16' | 'f.8' |
               'f.16'
               | 'bool' | 'void' | 'ptr'
               | IDENT

```

```

stmt          = var_decl | let_decl | if_stmt | while_stmt |
for_stmt      = return_stmt | 'break' | 'continue' | switch_stmt
              | asm_block | block | expr_stmt

var_decl      = 'var' IDENT ':' type at_clause? ('=' (array_init
| expr))?
let_decl      = 'let' (IDENT | '(' IDENT (',' IDENT)* ')') (':'
type)? '=' expr
array_init    = '[' expr (',' expr)* ']'

if_stmt       = 'if' expr block ('else' block)?
while_stmt    = 'while' expr block
for_stmt      = 'for' IDENT ':' type? 'in'
              (expr '[' expr? '..' expr? ']' block //
ForEachStmt (array)
              | expr '..' expr block) //
ForRangeStmt (int range)
return_stmt   = 'return' (expr | '(' expr (',' expr)* ')')?
switch_stmt   = 'switch' expr '{' case_clause* default_clause?
              '}'
case_clause   = 'case' INT ':' stmt*
default_clause = 'default' ':' stmt*
asm_block     = 'asm' IDENT? (('in' IDENT (',' IDENT)* ')')?
              '{' asm_line* '}'
block         = '{' stmt* '}'

expr_stmt     = expr ('=' expr)?

expr          = binary_expr
binary_expr   = unary_expr ((binop | 'as' type) binary_expr)*
binop         = '+' | '-' | '*' | '/' | '%' | '&' | '|' | '^'
              | '<<' | '>>' | '==' | '!=' | '<' | '<=' | '>' |
              '>='

unary_expr    = '-' unary_expr | '!' unary_expr | '~' unary_expr
              | '&' IDENT | postfix_expr

postfix_expr  = primary
              ( '^' // dereference
              | '[' expr ']' // index
              | '.' IDENT // field access
              | '.' IDENT '(' args ')' // UFCS method

call          = '(' args ')' // function

call          = '.map' '(' lambda ')' // iterator

chain         = '.filter' '(' lambda ')'
              | '.forEach' '(' lambda (',' expr)? ')'
              | '.fold' '(' expr ',' lambda ')'
              | '.reduce' '(' lambda ')'

```

```

        | '.take' '(' expr ')'
        | '.skip' '(' expr ')'
        | '.enumerate' '(' ')'
        | '.chain' '(' expr ')'
        | '.apply' '(' IDENT ')'
    )*

primary      = INT | 'true' | 'false'
LString      | STRING | 'c' STRING | 'l' STRING    // CString,
              | IDENT '.' IDENT                    // enum
access (State.IDLE)
              | ('u8'|'u16'|'i8'|'i16') '(' expr ')' // cast
              | 'sizeof' '(' type ')'               //
compile-time size
              | '@ptr' '(' type ',' expr ')'
              | '@print_u8' '(' expr ')' | '@print_nl' '(' ')' |
'@print_dec' '(' expr ')'
              | '@smc' IDENT ':' type               // SMC
parameter
              | 'range' '(' expr '..' expr ')'       // range
source
              | '|' lambda_params '|' (block | expr) // lambda
              | IDENT '{' field_init (',' field_init)* '}' //
struct literal
              | '(' expr (',' expr)* ')'            //
parenthesized / tuple
              | IDENT

lambda_params = (IDENT (':' type)? (',' IDENT (':' type)?))*?
lambda       = '|' lambda_params '|' (block | expr)

assert_expr  = IDENT '(' (INT (',' INT)*)? ')' '==' (INT | '('
INT (',' INT)* ')')
              ('via' ('mir2' | 'z80')))?

sandbox_block = 'sandbox' STRING '{' ('assert' assert_expr)* '}'

field_init   = IDENT ':' expr
args         = (expr (',' expr)*)?

```

Appendix B: Register Classes

MIR2 Register Classes

Class	Z80 register	Typical use	Cost (access)
ClassAcc	A		0T (ALU implicit)

Class	Z80 register	Typical use	Cost (access)
		Accumulator, u8 return, first param	
ClassCounter	B	DJNZ counter, second u8 param	4T (LD A,B)
ClassPointer	HL	u16 param/ return, pointer	0T (ADD HL implicit)
ClassIndex	DE	Second u16 param	4T (EX DE,HL to use in ADD)
ClassPair	BC	Third param, general pair	varies
ClassGeneral	C/D/E/H/L	Remaining 8- bit params	4T (LD A,r)
ClassDWord	HL+HL'	u32 via EXX shadow pair	~34T (ADD+EXX+ADC+EXX)
ClassFlag	F (flags)	Boolean return via carry/zero	0T at call, 4T materialize

Physical Register Availability

Register	Purpose in PBQP	Notes
A	ClassAcc	Cannot be used for indirect load to non-A
B	ClassCounter	DJNZ uses B implicitly
C	ClassGeneral	Most flexible 8-bit
D, E	ClassGeneral	DE pair for 16-bit address
H, L	ClassPointer (HL)	HL is the Z80's main 16-bit ALU reg

Register	Purpose in PBQP	Notes
IX, IY	ClassIndex	Struct field access (IX+d addressing)
HL', DE', BC'	ClassShadow, ClassDWord	EXX-accessed shadow pair
A'	ClassAccShadow	EX AF,AF'
F	ClassFlag	Carry = comparison result; no LD r,F — save via PUSH AF only

Memory-Backed Registers (\$F0xx)

When the interference graph has more simultaneously-live variables than physical registers, the allocator spills to absolute addresses in the \$F0xx range:

```
LD ($F001), A    ; spill - 13T
LD A, ($F001)    ; reload - 13T
```

Each round-trip costs 26T. The PreallocCoalesce pass (new in v4) reduces spills by unifying block-parameter registers.

Appendix C: CLI and Tools

Compiling Nanz

```
# Compile to Z80 assembly
mz source.nanz -o output.a80

# Compile and assemble to binary
mz source.nanz -o output.bin --assemble

# Compile to TAP (ZX Spectrum tape image)
mz source.nanz --target=spectrum -o game.tap

# Emit intermediate representations
mz source.nanz --emit-hir          # HIR dump
```

```

mz source.nanz --emit-mir2           # MIR2 before optimization
mz source.nanz --emit-mir2-opt       # MIR2 after optimization
mz source.nanz --emit-asm             # Z80 assembly

# Annotate T-states in output assembly
mz source.nanz --annotate-tstates -o annotated.a80

# Compile to native AMD64
mzn source.nanz                     # via QBE (default)
mzn -c source.nanz                  # via C99

```

The Toolchain

Tool	Binary	Description
MZC	mz	MinZ Compiler (Nanz/MinZ/PL/M-80 → Z80)
MZN	mzn	Native compiler (Nanz → AMD64 via C99/QBE)
MZA	mza	Z80 Assembler (table-driven, bracket syntax)
MZE	mze	Z80 Emulator (1335/1335 FUSE tests passing)
MZX	mzx	ZX Spectrum emulator (T-state accurate, AY sound)
MZD	mzd	Z80 Disassembler (IDA-like analysis, ABI propagation)
MZLSP	mzlsp	Language Server Protocol (diagnostics, hover, goto-def)
MZRUN	mzrun	Remote runner (DZRP protocol, for real hardware)
MZTAP	mztap	TAP file loader
MZV	mzv	MIR2 VM runner (breakpoints, tracing, PNG export)

Running Tests

```

cd minzc

# All packages
go test ./pkg/... -vet=off

# Specific test by name
go test ./pkg/nanz/ -run TestRangeFold_E2E_SumRange -v

```



```

# Z80 emulator tests
go test ./pkg/mir2/ -run TestMulU16ConstZ80 -v

# MOS 6502 E2E tests
go test ./pkg/mir2/ -run TestM6502 -v

# All iterator chain tests
go test ./pkg/nanz/ -run TestRange -v

```

Appendix D: What's New in v4

Features shipped since v3 (2026-03-11):

Feature	Chapter	Status
expr as type cast syntax	2.9	Shipped
Signed comparison (i8/i16)	3.2	Shipped
PreallocCoalesce	8.5	Shipped — 6 showcase files improved
Trivial inliner	8.6	Shipped — swap→RET, min_of→EQU
ForEachEdge visitor	8.1	Shipped — ~75 LOC removed
mzn native compiler	13	Shipped
MOS 6502 backend	9.2	Shipped — 35/35 tests
VSCode native compilation	13.3	Shipped
BUG-003 fix (ptr[i] in while)	—	Fixed (5 interacting codegen bugs)
BUG-006 fix (zero-size globals)	—	Fixed (bare label emission)
BUG-007 fix (spurious adapter LD)	—	Fixed (identity copy skip)
	8.7	

Feature	Chapter	Status
Multi-pass contract nudges		Shipped (mul16 rhs→DE, DJNZ→B)

Assembly improvements (v3 → v4)

Example	Before	After	Saving
ex7_mapInPlace	5-inst loop back-edge	1 DJNZ	4 insts, ~30T/iter
ex6_forEach/ max_chain	trampoline + 3 insts	AND A + JRS Z	trampoline eliminated
ex9b_factorial_fold	56 lines + mul16	32 lines	mul16 routine gone
ex10b_fib_iter	3× EX DE,HL	ADD HL,HL	3 insts removed
ex10c_fib_fold	8 register shuffles	2 moves	6 moves removed
ex9a_factorial_rec	n=B contract	n=A contract	natural ABI

Appendix E: What's New in v4.1

Features shipped since v4 (2026-03-13):

Feature	Chapter	Status
sizeof(Type) compile-time operator	2.15	Shipped — all primitives + user structs
sandbox blocks for shared-VM assertions	7.7	Shipped — MIR2 VM + Z80 emulator
Arena allocator pattern	15	Shipped — init, alloc, reset, remaining, arena_split
Lifetime tiers (perm/level/frame)	15.4	Shipped — documented pattern
ConstantCallElim fix	8.1	

Feature	Chapter	Status
		Fixed — calls with all-const args now fold correctly
Showcase count	11.8	24/24 (was 23/23)

New syntax summary (v4 → v4.1)

Syntax	Meaning	Resolved at
<code>sizeof(Type)</code>	Size of type in bytes	Parse time
<code>sandbox "name" { ... }</code>	Shared-VM assertion group	Compile time

Appendix F: What's New in v5

Features shipped since v4.1 (2026-03-14):

Feature	Chapter	Status
Enum declarations	16.1	Shipped — auto-increment + explicit values, dot access
Type aliases	16.2	Shipped — structural aliases (type <code>Score = u16</code>)
Module system	17	Shipped — unqualified, qualified, alias, glob imports
SString (u8-prefix)	18.1	Shipped — default string type
LString (u16-prefix)	18.1	Shipped — <code>l"..."</code> prefix
CString (NUL-terminated)	18.1	Shipped — <code>c"..."</code> prefix
Triple-quote strings	18.1	Shipped — <code>"""..."""</code> multi-line
String interpolation	18.4	

Feature	Chapter	Status
		Shipped — <code>#{expr}</code> with compile-time folding
StringPool dedup	18.2	Shipped — identical strings share storage
Pipe/trans declarations	19.2	Shipped — named reusable pipelines
Pipeline composition (use)	19.2	Shipped — snapshot semantics
<code>.apply()</code>	19.3	Shipped — connect pipe to data source
DJNZ pipe fusion	19.4	Shipped — all stages inline into loop body
<code>\ ></code> prefix syntax	19.5	Shipped — optional visual prefix in pipe body
<code>@derive_debug(Type)</code>	20.2	Shipped — print all struct fields
<code>@derive_eq(Type)</code>	20.2	Shipped — field-by-field equality
<code>@derive_sizeof(Type)</code>	20.2	Shipped — sizeof + per-field offsets
MetaRuntime introspection	20.4	Shipped — VM host functions for type/struct/AST
Cross-language <code>.lanz</code> import	21.2	Shipped — S-expression modules
Cross-language <code>.plm</code> import	21.3	Shipped — PL/M-80 legacy modules
Circular import detection	21.5	Shipped — stack-based cycle check
Showcase count	11	28/28 (was 24/24)

New syntax summary (v4.1 → v5)

Syntax	Meaning	Resolved at
<code>enum Name { A, B = 5 }</code>	Named integer constants	Parse time
<code>type Alias = ExistingType</code>	Structural type alias	Parse time
<code>import mod.sub { sym }</code>	Module import	Compile time
<code>"text" / l"text" / c"text"</code>	SString / LString / CString	Compile time
<code>"""multi\nline"""</code>	Triple-quote string	Compile time
<code>@print("#{expr}")</code>	String interpolation	Compile time (const) / Runtime
<code>pipe name { map(λ); ... }</code>	Named iterator pipeline	Compile time
<code>trans name { use other; ... }</code>	Pipeline composition	Compile time
<code>source.apply(pipe)</code>	Apply pipe to data	Compile time (fused)
<code>\ > stage(...)</code>	Optional pipe stage prefix	Parse time
<code>@derive_eq(Type)</code>	Generate equality function	Compile time
<code>@derive_debug(Type)</code>	Generate debug print function	Compile time
<code>@derive_sizeof(Type)</code>	Generate sizeof + offsetof functions	Compile time
<code>import mod { sym } (.lanz)</code>	Import Lanz S-expression module	Compile time
<code>import mod { sym } (.plm)</code>	Import PL/M-80 module	Compile time

Feature gap closed

Six of the eight features that were “only in MinZ” have been ported to Nanz:

Feature	v4.1 Status	v5 Status
Enums	MinZ only	Nanz (Chapter 16)

Feature	v4.1 Status	v5 Status
String interpolation	MinZ only	Nanz (Chapter 18)
Import system	MinZ only	Nanz (Chapter 17)
Type aliases	Not implemented	Nanz (Chapter 16)
Pipe/trans pipelines	Not implemented	Nanz (Chapter 19)
@derive metafunctions	Not implemented	Nanz (Chapter 20)
Cross-language imports	Not implemented	Nanz (Chapter 21)
@error propagation	MinZ only	MinZ only (next priority)
@define macros	MinZ only	MinZ only
@if/@elif conditionals	MinZ only	MinZ only

Appendix G: What's New in v5.2

Features shipped since v5 (2026-03-15):

Feature	Chapter	Status
ptr(addr) cast	2.15	Shipped — u16→ptr, language-level peek/poke
ptr(addr)^ = val lvalue	2.15	Shipped — direct memory write without asm
\ > value pipe operator	2.16	Shipped — F#/Elixir-style function chaining
(ret REG) asm clause	2.14	Shipped — explicit return register
(out REG) asm clause	2.14	Shipped — alias for ret
(clob REG, ... \ auto\ all)	2.14	Shipped — precise clobber specification

Feature	Chapter	Status
(in REG) register-style	2.14	Shipped — register names, auto-infer default
Auto-clobber analysis	2.14	Shipped — parse asm text, compute write-set
Lambda type inference	5	Shipped — \ x\ x + x without : u8 annotation
ZX Spectrum Tetris	—	853 LOC Nanz → 2176 lines Z80 asm
Showcase count	11	34/34 (was 28/28)

New syntax summary (v5 → v5.2)

Syntax	Meaning	Resolved at
<code>ptr(addr)^</code>	Read byte at address	Compile time (cast is no-op)
<code>ptr(addr)^ = val</code>	Write byte to address	Compile time
<code>expr \ > f</code>	<code>f(expr)</code>	Parse time (desugars to call)
<code>expr \ > f(a)</code>	<code>f(expr, a)</code>	Parse time
<code>asm z80 (ret A) { ... }</code>	Asm with return register	Compile time
<code>asm z80 (clob A, F) { ... }</code>	Explicit clobber list	Compile time
<code>asm z80 (clob auto) { ... }</code>	Auto-detect clobbers	Compile time
<code>asm z80 (in HL) { ... }</code>	Register-style input	Compile time

Tetris: Complete Game in Nanz

853 lines of Nanz compile to a playable Tetris for ZX Spectrum 48K: - 7 tetrominoes with SRS-lite wall kicks - Hold piece, next piece preview, ghost piece - T-spin detection with bonus scoring - Attribute-based rendering (fast — only color bytes change per frame) - 48 Z80 functions, 2176 lines of assembly

```
cd minzc && ./mz ../examples/zx/tetris.nanz -o /tmp/tetris.a80
./mza /tmp/tetris.a80 -o /tmp/tetris.bin && ./mzx --run /tmp/
tetris.bin@8000
```

Appendix H: What's New in v5.3

Features shipped since v5.2 (2026-03-15):

Feature	Chapter	Status
Five-frontend architecture	21	Nanz, Lanz, Lizp, PL/M-80, Pascal
Pascal frontend	21.2, 21.6	Turbo Pascal → HIR → Z80 (CP/M target)
Lizp frontend with macros	21.2, 21.4	defmacro, threading, desugars to Lanz
Universal compile-time assert	21.9	Same hir.Assert pipeline in all 5 frontends
Pascal → CP/M hello world	21.6	WriteLn → BDOS ConOut via inline asm
Cross-frontend bug detection	21.11	Same algorithm, five syntaxes → catches parser bugs
Transpilation (--emit)	21.10	Convert between any pair of frontends
.lizp cross-language import	21.4, 21.12	import macrolib { lizp_double }
.pas cross-language import	21.6, 21.12	import pascal_math { DOUBLE }

Five Frontends, One Pipeline

The compiler now supports five source languages, all converging on the same $\text{HIR} \rightarrow \text{MIR2} \rightarrow \text{Z80}$ pipeline. A function `double(x) = x + x` written in any of the five languages produces the same Z80 output:
`ADD A, A / RET.`

The compile-time assert system — dual-VM verification on both MIR2 VM and Z80 binary — works identically across all frontends. This was validated with 9 assert tests per language (`double`, `add`, `max_byte`) across Nanz, Lanz, Lizp, PL/M-80, and Pascal.

Pascal Hello World on CP/M

```
program Hello;
begin
  WriteLn('Hello from Pascal on Z80!');
end.

mz hello.pas -t cpm -o hello.com
mze -t cpm hello.com
# Output: Hello from Pascal on Z80!
```

The Pascal lowerer generates CP/M BDOS wrappers (`ConOut`, `WriteStr`, `WriteCrLf`) directly as HIR functions with inline Z80 asm, ensuring correct register placement (C=function, DE=parameter, CALL \$0005).

Appendix I: What's New in v5.4

Features shipped since v5.3 (2026-03-17):

Feature	Location	Status
FAT12/16 R/W library	stdlib/fs/fat12.minz	mount, find, read, create, delete, overwrite
FAT12 write_fat12	12-bit packed R-M-W	Round-trip verified (5 asserts)
FAT16 support	Auto-detect by cluster count	read_fat16/ write_fat16 + unified dispatch

Feature	Location	Status
Bidirectional FatFS testing	pkg/c89/fatfs_vm_test.go	gcc→MIR2 (5/5), MIR2→gcc (7/7)
Nanz write verification	TestNanzFAT12_Write	13/13 subtests + gcc 14/14 cross-verify
E2E 5-channel verification	TestE2E_NanzWrite_MultiChannelVerify	Nanz VM, fresh VM, gcc, C89 MIR2, raw bytes
SDCC Z80 comparison	TestDifferential_Z80_vs_SDCC	Per-function instruction counts vs SDCC bytes
C89→QBE native path	pkg/c89/fatfs_vm_test.go	33/33 FatFS low-level asserts via QBE
Differential code quality	pkg/c89/fatfs_differential_test.go	Nanz MIR2 99 vs C89 97 instr (+2.1%)
C89 do-while + break/continue	pkg/c89/lower.go	19 asserts
QBE OpAdd l-typed promotion	pkg/mir2qbe/codegen.go	Pointer arithmetic fix
C89 corpus expanded	16 files, 350 asserts	+2 files, +159 asserts

FAT12/16 Filesystem Library — `stdlib/fs/fat12.minz`

Full read-write FAT filesystem library in idiomatic Nanz. Supports FAT12 and FAT16 volumes with automatic type detection at mount time. Designed for embedded/retro targets (Z80, eZ80, 6502).

Read API: `fat_mount`, `find_file`, `file_read`, `read_named_file`, `count_dir_entries`, `get_dir_entry`

Write API: `create_file`, `delete_file`, `overwrite_file`, `fat_sync`

Internal: `write_fat12` (12-bit packed read-modify-write), `write_fat16`, `alloc_cluster`, `free_chain`, dirty-tracking sector window + FAT cache with write-back to all FAT copies.

```
fun write_fat12(fat: ^u8, clst: u16, val: u16) -> void {
  let half: u16 = clst >> 1
  let ofs: u16 = clst + half
  let raw: u16 = ld_word(fat + ofs)
  let odd: u16 = clst & 1
  var new_raw: u16 = 0
  if odd != 0 {
    let keep: u16 = raw & 0x000F
    let shifted: u16 = val << 4
    new_raw = keep | shifted
  } else {
    let keep: u16 = raw & 0xF000
    let masked: u16 = val & 0x0FFF
    new_raw = keep | masked
  }
  st_word(fat + ofs, new_raw)
}
```

Verified end-to-end via **5 independent channels**: Nanz writes text files, binary files (0xDEADBEEF pattern), multi-sector files (700B, i%251), deletes and overwrites — then verified by (A) same Nanz VM, (B) fresh Nanz VM reload, (C) gcc-compiled FatFS R0.16 (14/14), (D) C89 MIR2 VM FAT structure, (E) raw byte inspection. FAT copy synchronization verified. Differential testing proves 28/28 bit-identical low-level results vs C89. 11 total FatFS tests, all PASS.

Chapter 22: Self-Hosting — Can Nanz Compile Itself?

The holy grail of language design: a compiler written in its own language. Can Nanz compile Nanz? The short answer is *partially yes*, and the architecture makes this more interesting than a simple “write a parser in itself” exercise.

22.1 The Staged Pipeline Vision

The MinZ compilation pipeline has natural **stage boundaries**:

Stage 1: Source → HIR	(parsing, name resolution, type checking)
Stage 2: HIR → MIR2	(lowering to SSA, typed virtual registers)
Stage 3: MIR2 → MIR2	(optimization passes: DCE, const fold, Grace rules)
Stage 4: MIR2 → Z80 ASM	(register allocation, instruction selection, peephole)
Stage 5: ASM → Binary	(assembly, label resolution, relocation)

Each stage is a pure function: `data in → data out`. This means each stage could be a **separate tool**, and any single stage could be rewritten in Nanz while the others remain in Go.

The key insight: **you don't need to self-host the entire compiler at once**. You can self-host one stage at a time, using the Go compiler to bootstrap the rest.

22.2 What's Feasible Today

Stage 5 (Assembler) — Feasible Now

The Z80 assembler (`mza`) is table-driven: opcode table + label resolution + binary emit. This is ~8KB of logic with no complex data structures. A Nanz implementation could: - Use a fixed-size array for labels (512 entries covers most programs) - Walk instruction tokens linearly - Emit binary bytes to a buffer

This fits in 48KB and could run on a real Z80 under CP/M.

Stage 4 (Codegen) — Feasible with Effort

The Z80 code generator reads MIR2 (virtual registers + typed ops) and emits assembly. It's ~15KB of pattern matching logic. With `match` expressions now available, this maps naturally to Nanz:

```
enum MirOp { Add, Sub, Mul, Load, Store, Call, Cmp, Br, Ret }

fun emit_op(op: MirOp, dst: u8, src1: u8, src2: u8) {
    match op {
        Add => emit_add(dst, src1, src2),
        Sub => emit_sub(dst, src1, src2),
        Load => emit_load(dst, src1),
        _ => emit_generic(op, dst, src1, src2),
    }
}
```

```
}  
}
```

Stage 3 (Optimizer) — Partially Feasible

Individual optimization passes are small, self-contained functions. Dead store elimination, constant folding, peephole — each is 200-500 lines. These could be written in Nanz as separate tools that read and write MIR2 text format.

22.3 What's Hard

Stage 1 (Parser) — The Big Challenge

The Nanz parser (`parse.go`) is 4700+ lines of Go with: - Recursive descent (deep call stacks) - Hash maps for symbol tables (1000+ entries for `stdlib`) - String manipulation (identifier names, error messages) - Dynamic AST construction

Nanz lacks: hash maps, dynamic strings, deep recursion support. A self-hosted parser would need: - Linear-probing hash table on a fixed `[u16; 1024]` array - Identifier interning via offset into a pre-allocated byte buffer - Iterative parsing (convert recursion to explicit stack)

Estimated size: ~50KB of Nanz code + ~30KB working memory = doesn't fit in 48KB Z80 memory.

But it could run on MZV — the MIR2 VM has 64KB heap and configurable gas limits. A Nanz parser running on MZV is architecturally equivalent to a cross-compiler: the parser runs on the host (via MZV), producing MIR2 that targets Z80.

22.4 The MZV Path

MZV (the MIR2 VM runner) already proves that complex Nanz programs can execute: Tetris runs, interactive demos work, the FAT filesystem library processes real disk images. A self-hosted compiler stage running on MZV is not hypothetical — it's the same execution model.

The missing pieces for MZV-hosted compilation: 1. **File I/O host functions** — `@mir.io.read_file(path), @mir.io.write_file(path, data)`. Currently MZV only has print I/O. Adding file ops is straightforward Go. 2. **String operations** — at minimum, string comparison and substring extraction. Can be implemented as host

functions or as Nanz library code operating on byte buffers. 3. **Larger heap** — 64KB default is tight for a compiler. MZV's heap is configurable; 256KB or 1MB would suffice.

22.5 TinyNanz: A Bootstrapping Subset

One practical approach: define **TinyNanz** — a minimal subset of Nanz that can express a parser:

Feature	TinyNanz	Full Nanz
Types	u8, u16, ^u8	u8, u16, i8, i16, bool, structs, arrays
Control flow	if/else, while, return	+ for, match, switch, break/continue
Functions	fun, no overloading	+ overloading, lambdas, UFCS
Data	global arrays, pointers	+ structs, enums, ADTs
Strings	byte buffers + length	+ interpolation, 3 string types

A TinyNanz-to-MIR2 compiler in TinyNanz would be ~20KB — comfortably fits on Z80. It couldn't compile full Nanz, but it could compile *itself*, achieving true self-hosting for the subset.

22.6 The Multi-Tool Architecture

Instead of one monolithic compiler binary, imagine:

```
# Each stage is a separate tool, each written in Nanz
nanz-parse program.nanz -o program.hir      # Stage 1: Source → HIR
nanz-lower program.hir -o program.mir       # Stage 2: HIR → MIR2
nanz-opt program.mir -o program.opt.mir     # Stage 3: Optimize
nanz-codegen program.opt.mir -o program.a80 # Stage 4: Codegen
mza program.a80 -o program.com              # Stage 5: Assemble
```

Benefits: - Each tool is small enough to run on Z80 or MZV - Each tool can be tested independently - You can mix Go and Nanz tools in the pipeline - Self-hosting progresses one stage at a time - The pipeline becomes a **build system**, not a monolith

22.7 Error Handling: @error vs ADT Result

Nanz has two complementary error handling approaches:

@error + CY flag — Z80-native, zero overhead:

```
fun read_byte?(addr: u16) -> u8 ? ErrCode {
    if addr == 0 { @error(ErrCode.NotFound) } // SCF + LD A,
errcode + RET
    return ptr(addr)^ // OR A + RET
(clear CY)
}
// Caller: CALL read_byte → JR C, .handle_error (ONE instruction)
```

ADT Result — portable, composable:

```
enum Result { Ok(u8), Err(u8) }

fun safe_add(a: u8, b: u8) -> u16 {
    if (u16(a) + u16(b) > 255) { return Err(1) }
    return Ok(a + b)
}
// Caller: __tag(result) check (2-3 instructions)
```

Approach	Check cost	Z80-native?	Composable?	Payload size
@error + CY	1 instruction (JR C)	Yes	No (CY is single bit)	u8 (A register)
ADT Result	2-3 instructions	Via u16 HL	Yes (chain, map)	u8

Future direction: @error_abi annotation that maps ADT Result to CY flag calling convention, getting the best of both worlds — composable Rust-style Result syntax with Z80-native single-instruction error checking.

22.8 Roadmap to Self-Hosting

Phase	Scope	Runs on	Effort
0 (done)	MZV runs complex Nanz (Tetris, FAT)	Host via Go	Done
1		Z80 / CP/M / Agon	2-4 weeks

Phase	Scope	Runs on	Effort
	Nanz Z80 assembler (Stage 5)		
2	Nanz MIR2→Z80 codegen (Stage 4)	MZV / Agon	1-2 months
3	Nanz optimizer passes (Stage 3)	MZV / Agon	1-2 months
4	TinyNanz parser (Stage 1 subset)	MZV / Agon / Spectrum 128K	2-3 months
5	Full Nanz parser (Stage 1)	MZV / Agon	3-6 months
6	Native self-host on Agon Light 2	Agon (512KB, 18MHz eZ80)	1-2 months after Phase 5

Phase 1-2 deliver real value: a Nanz-written backend that produces Z80 code, bootstrapped by the Go frontend. Phase 4-5 close the loop. Phase 6 is the prize: Nanz compiling Nanz on real eZ80 hardware — a compiler that runs on the machine it targets.

The honest assessment: Self-hosting on a stock 48KB ZX Spectrum is tight — the compiler alone is 80-120KB. But several real hardware targets make native self-hosting practical:

Platform	Available RAM	Feasibility
ZX Spectrum 48K	42KB usable	Too tight for full compiler
ZX Spectrum 128K	128KB (8 banks)	Feasible — staged compilation across banks
Agon Light 2 (eZ80)	512KB	Easy — entire compiler fits in flat memory
CP/M + banked RAM	256KB+ (Z180, CPC6128, MSX2)	Feasible — disk swap for large programs

Platform	Available RAM	Feasibility
MZV (MIR2 VM)	Configurable (64KB–16MB)	Easy — no hardware constraints

On Spectrum 128K, the multi-tool architecture maps naturally to bank switching: parser in banks 0-2, optimizer in banks 3-4, codegen in banks 5-7. Each stage reads input from a shared buffer in the unbanked 32KB region and writes output back. The `@target(spectrum128)` annotation could even generate bank-switching trampolines automatically.

On Agon Light 2 with 512KB flat RAM and 18MHz eZ80, the entire compiler fits comfortably with room for 300KB+ of source code and working memory. This is the most natural native self-hosting target.

MZV remains the lowest-friction path — no hardware constraints, easy debugging, extensible host functions — but native Z80/eZ80 self-hosting is not a dream, it's an engineering exercise on the right hardware.

Appendix J: What's New in v7.0

VIR: Z3 SMT Backend (Default)

The compiler now uses a **Z3 SMT solver** for joint instruction selection and register allocation. Instead of heuristic graph coloring (PBQP), VIR encodes the entire allocation problem as a satisfiability formula and solves it optimally.

Pipeline: HIR → MIR2 → VIR (Z3 solver) → Z80 assembly

	—	Z3-PFCC0: optimal calling conventions
	—	ISLE combining: load fusion, MUL
strength reduction	—	CFG-aware: cross-block register
correctness	└—	16 peephole rules

- **55 Z80-verified asserts**, 496/496 pipeline coverage
- **5/5 SDCC wins** on benchmark functions
- PBQP fallback for functions with inline assembly
- Inline `div8/mod8/mul8` runtime routines per call site

impl Blocks

See [Chapter 4.4](#). Group interface implementations:

```
impl Shape for Circle {  
    fun area(self) -> u8 { return 3 * self.radius * self.radius }  
}
```

Wider Integer Types

Type	Width	Target
u24 / i24	24-bit	eZ80 / Agon Light 2 (native)
u32 / i32	32-bit	MZV VM, Z80 shadow registers

Supported in declarations, casts (`x as i32`), and function-style casts (`i32(x)`).

Local Array Literals

```
let data: [u8; 5] = [10, 20, 30, 40, 50]
```

Generates a mangled global (`__arr_N`) with the literal data. The local variable binds to its address. On Z80 there are no stack-allocated arrays — this is the natural encoding.

else-if Chains

```
if x == 0 {  
    ...  
} else if x == 1 {  
    ...  
} else {  
    ...  
}
```

Stream Abstraction (`stdlib/core/stream.nanz`)

Unified write interface:

```
global buf: [u8; 128]  
var s: BufStream  
bufstream_init(&s, &buf, 128)  
bufstream_write_u8(&s, 72)    // 'H'  
bufstream_write_u8(&s, 105)   // 'i'  
// buf = "Hi", bufstream_pos(&s) = 2
```

Three backends: BufStream (memory), NullStream (discard/count), Stdout (platform I/O — planned).

S-Expression Frontends: Lambda, Match, Let-in

Lanz and Lizp gained functional programming primitives:

```
;; Lambda
(defun test () -> u8
  (return (apply (fn ((x u8)) u8 (return (+ x x))) 5)))

;; Scoped let-in
(defun f ((x u8)) -> u8
  (return (let* ((a u8 (+ x 1)) (b u8 (* a 2))) b)))

;; Pattern match
(defun classify ((x u8)) -> u8
  (return (case x (0 10) (1 20) (_ 99))))
```

MinZ Corpus Convergence

58/119 legacy MinZ files (49%) now parse through the Nanz pipeline.
Changes: - let mut replaced with var across corpus - Trailing semicolons removed (Nanz has no semicolons) - *u8 pointer syntax replaced with ^u8

9 Frontends, One Pipeline

Frontend	Extension	Style
Nanz	.nanz	Rust-like, primary
Frill	.frl	ML/Haskell functional
Lizp	.lizp	Scheme/Lisp with macros
Lanz	.lanz	S-expression HIR
C89	.c	C89 subset
PL/M-80	.plm	Intel PL/M-80
Pascal	.pas	Pascal subset
ABAP	.abap	SAP ABAP subset
MinZ	.minz	Legacy (49% compat)

All route through: Frontend → HIR → MIR2 → VIR/PBQP → Z80 assembly.

Appendix K: What's New in v8.0 (Birthday Marathon)

@error — Z80-Native Error Propagation

CY flag + A register. The Z80 was designed for this pattern.

```
fun safe_div?(a: u8, b: u8) -> u8 {
  if b == 0 { @error(1) }      // SCF / LD A, 1 / RET - 2 bytes
  return a / b
}

fun compute(a: u8, b: u8) -> u8 {
  var x: u8 = safe_div?(a, b)
  @propagate                    // RET C - 1 byte!
  return x + 1
}
```

Layer 2 enforcement: ? in function name = fallible. Compiler **requires** @check/@propagate after every ?-call. Missing it → compile error. Zero runtime overhead.

Z80 codegen: - @error(N) → SCF / LD A, N / RET (set carry, error code, return) - @propagate → RET C (1 byte! conditional return on carry) - @check → JR NC, .ok / RET / .ok: (check + propagate inline)

VIR as Default Backend

VIR (Z3 SMT solver) is now the default. --vir=true by default, --lir for legacy.

Pipeline: Source → HIR → MIR2 → VIR (Z3 optimal) → PBQP fallback → Z80 ASM

Z3 mathematically proves optimal register allocation. Example — abs_diff:

```
fun abs_diff(a: u8, b: u8) -> u8 {
  if a > b { return a - b }
  return b - a
}
```

Z3 output (provably optimal, 4 bytes):

```
abs_diff:      ; params: a=A, b=C (PFCC0)
  SUB C        ; a - b, sets carry if b > a
  RET NC       ; if a >= b, return a-b
```

```

NEG          ; else negate: -(a-b) = b-a
RET          ; 4 bytes – hand-optimal!

```

Compare with typical hand-written (6+ bytes):

```

; hand-written abs_diff (typical)
CP C          ; compare a, b
JR NC, .ok    ; if a >= b, skip
LD A, C       ; a = b
SUB (saved)   ; ...complex
.ok:
SUB C
RET

```

Z3 found SUB/RET NC/NEG/RET — shorter than most hand-written versions.

Five Backends — Z80 to GPU

Same Nanz source compiles to 5 targets:

```

mz program.nanz -b z80      -o out.a80      # Z80 (1976)
mz program.nanz -b cuda     -o out.cu       # NVIDIA CUDA
mz program.nanz -b opencl   -o out.cl       # AMD/Intel OpenCL
mz program.nanz -b vulkan    -o out.comp     # Vulkan GLSL
mz program.nanz -b metal     -o out.metal    # Apple Metal

```

All 4 GPU backends verified 256/256 on real hardware (NVIDIA, AMD RX 580, Apple M2).

Nanz and Frill produce identical output — both lower to the same MIR2:

	Nanz	Frill
Source	fun double(x: u8) -> u8 { return x + x }	let double (x : u8) : u8 = x + x
Z80	ADD A, A / RET	ADD A, A / RET
CUDA	r2 = (r1 + r1) & 0xFF;	r2 = (r1 + r1) & 0xFF;

Choose your syntax — Swift-like (Nanz) or ML-like (Frill) — get the same optimal code.

BCD Packed Decimal Types

New type family for COBOL/financial arithmetic:

```

var price: bcd8 = 42          // stored as 0x42, not 0x2A
var tax: bcd8 = 10           // stored as 0x10
// Z80: ADD A, B / DAA – decimal adjust after add (4T extra)

```

Types: bcd8 (2 digits), bcd16 (4 digits), bcd24 (6 digits), bcd32 (8 digits).
Big-endian BCD (COBOL/IBM convention).

RLCA Sled — Barrel Shifter (9 bytes)

Multi-entry function: 8 entry points for all rotation counts.

```

__rotate_7:  RLCA      ; 8 entry points, fall-through cascade
__rotate_6:  RLCA
__rotate_5:  RLCA
__rotate_4:  RLCA      ; ← nibble swap entry
__rotate_3:  RLCA
__rotate_2:  RLCA
__rotate_1:  RLCA
__rotate_0:  RET       ; 9 bytes total

```

CALL __rotate_4 = nibble swap. Assembly peephole auto-folds 3+ consecutive RLCA's.

MZA INCBIN — Binary Data Embedding

```

sprite_data:  INCBIN "player.spr"
font_8x8:     INCBIN "font.bin", 0, 768
mul_table:    INCBIN "mulopt8.bin"

```

Embed binary files directly in assembly. With #embed (C23) for C frontend.

Codegen Quality: Z3 vs Hand-Written

Function	Z3 (VIR)	Hand-written	Winner
abs_diff	4 bytes (SUB/RET NC/NEG/RET)	6+ bytes	Z3
popcount	7 insts (LUT O(1))	7 insts (same)	tie
double	1 inst (ADD A,A)	1 inst (ADD A,A)	tie
safe_div	SCF/RET + body	SCF/RET + body	tie
swap	0 insts (PFCCO)	20 insts (SDCC)	Z3 (20:0!)

Z3 is **at least as good** as hand-written for leaf functions, and **dramatically better** for calling conventions (PFCCO eliminates parameter passing overhead).

GPU-Precomputed Arithmetic (z80-optimizer v1.0.0)

501 provably optimal sequences embedded in the compiler: - 254/254 constant multiplies (mul8) - 246/247 constant divisions (div8) - 83.6M exhaustive register allocations ($\leq 6v$) - 4.4M dead-flags peephole rules - RLCA sled, branchless ABS, branchless NOT

1046 Total Assertions

Corpus	Asserts
Nanz examples (35)	35/35 compile
C89 corpus (38 files)	350 mir2
C99+ corpus (19 files)	269 mir2
Frill examples (16 files)	427 compile-time
Total	1046

MinZ v0.23.0 — Birthday Marathon Release. 8 frontends, 5 backends, 50 years of hardware. “The compiler never fails. It only varies in how optimal the result is.” <https://github.com/oisee/minz>